UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO – DCA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

ELDER SANTOS SILVA

PLATAFORMA WEB PARA GERENCIAMENTO E EXECUÇÃO DE AUTOMAÇÕES CONTÁBEIS COM APLICAÇÃO PRÁTICA EM ESCRITÓRIO PARCEIRO

NATAL – RN
2025

ELDER SANTOS SILVA

PLATAFORMA WEB PARA GERENCIAMENTO E EXECUÇÃO DE AUTOMAÇÕES CONTÁBEIS COM APLICAÇÃO PRÁTICA EM ESCRITÓRIO PARCEIRO

Monografia apresentada ao curso de graduação em Engenharia de Computação, da Universidade Federal do Rio Grande do Norte, como requisito parcial à obtenção do título de Bacharel em Engenharia de Computação.

Orientador: Prof. Dr. Diogo Fernandes Pinheiro Pedrosa.

NATAL – RN

2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Silva, Elder Santos.
Plataforma web para gerenciamento e execução de automações contábeis com aplicação prática em escritório parceiro / Elder Santos Silva. - 2025.
64 f.: il.

Monografia (graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia de Computação, Natal, RN, 2025.
Orientação: Prof. Dr. Diogo Pinheiro Fernandes Pedrosa.

1. Automação Robótica de Processos - RPA - Monografia. 2. Engenharia de Software - Monografia. 3. Automação de processos - Monografia. I. Pedrosa, Diogo Pinheiro Fernandes. II. Título.

RN/UF/BCZM CDU 004.896

Elaborado por Fernanda de Medeiros Ferreira Aquino - CRB-15/301

ELDER SANTOS SILVA

PLATAFORMA WEB PARA GERENCIAMENTO E EXECUÇÃO DE AUTOMAÇÕES CONTÁBEIS COM APLICAÇÃO PRÁTICA EM ESCRITÓRIO PARCEIRO

Monografia apresentada ao curso de graduação em Engenharia de Computação, da Universidade Federal do Rio Grande do Norte, como requisito parcial à obtenção do título de Bacharel em Engenharia de Computação.

Aprovada em: 24/07/2025

BANCA EXAMINADORA

Prof. Dr. Diogo Pinheiro Fernandes Pedrosa
Orientador
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Prof. Me. Micael Balza
Membro interno
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Me. Samuel Alves da Costa
Membro externo
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE – UFRN

AGRADECIMENTOS

A Deus, pela saúde e força que me permitiram concluir esta etapa.

À minha família, por todo o suporte e por me proporcionarem as condições para que eu pudesse me dedicar aos estudos. Esta conquista é nossa.

À minha companheira, Heloise, pela parceria constante nos momentos bons e ruins. Sua presença e seu apoio foram essenciais em toda a caminhada.

Ao meu orientador, Prof. Dr. Diogo Pinheiro, por aceitar me guiar na construção deste trabalho, compartilhando seus valiosos conhecimentos e sua orientação precisa.

À Universidade Federal do Rio Grande do Norte (UFRN), minha segunda casa durante a graduação, por todo o aprendizado e pela estrutura acolhedora que foi fundamental para minha formação.

RESUMO

Este trabalho apresenta o desenvolvimento de uma plataforma web para gerenciamento e execução de automações contábeis, visando otimizar a rotina de escritórios de pequeno porte. O objetivo principal foi construir uma solução robusta e escalável que, além de executar robôs de software baseados em Automação Robótica de Processos (*Robotic Process Automation* – RPA), oferecesse uma infraestrutura centralizada para monitoramento, manutenção e governança, aplicando boas práticas de Engenharia de Software. A metodologia adotada incluiu o levantamento de requisitos em parceria com um escritório contábil, a concepção de uma arquitetura de microsserviços implantada na nuvem (Google Cloud Platform) e o desenvolvimento dos componentes com tecnologias como Python, Django, FastAPI e Docker. Como prova de conceito, automatizou-se o processo de consulta ao Domicílio Tributário Eletrônico (DTE) na Unidade Virtual de Tributação (UVT) do Rio Grande do Norte. Os resultados, validados ao longo de três semanas com uma base de 92 clientes, demonstraram a estabilidade da plataforma após ajustes iniciais de infraestrutura e a eficácia da automação, que gerou uma economia de tempo estimada em 10 horas de trabalho manual. Conclui-se que os objetivos foram atingidos, entregando um Produto Mínimo Viável (MVP) funcional que validou a arquitetura proposta e demonstrou o potencial da solução para aumentar a eficiência operacional, sendo sugeridos para trabalhos futuros o agendamento de execuções e a implementação de notificações.

Palavras-chave: Automação de processos; RPA; Engenharia de Software; Computação em nuvem; Contabilidade.

ABSTRACT

This work presents the development of a web platform for managing and executing accounting automations, aimed at optimizing the daily operations of small-sized firms. The primary objective was to build a robust and scalable solution that, in addition to executing software robots based on Robotic Process Automation (RPA), provided a centralized infrastructure for monitoring, maintenance, and governance, applying Software Engineering best practices. The adopted methodology included requirements elicitation in partnership with an accounting firm, the design of a microservices architecture deployed on the cloud (Google Cloud Platform), and the development of components using technologies such as Python, Django, FastAPI, and Docker. As a proof of concept, the process of querying the Electronic Tax Domicile (DTE) within the Virtual Taxation Unit (UVT) of the state of Rio Grande do Norte was automated. The results, validated over three weeks with a base of 92 clients, demonstrated the platform's stability following initial infrastructure adjustments and the effectiveness of the automation, which yielded estimated time savings of 10 hours of manual labor. It is concluded that the objectives were achieved, delivering a functional Minimum Viable Product (MVP) that validated the proposed architecture and demonstrated the solution's potential to increase operational efficiency. For future work, the scheduling of executions and the implementation of notifications are suggested.

Keywords: Process automation; RPA; Software Engineering; Cloud computing; Accounting.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura da solução 30
Figura 2 – Tela de login 35
Figura 3 – Tela inicial para o usuário Desenvolvimento/Suporte 36
Figura 4 – Tela inicial para o usuário Gestor 37
Figura 5 – Tela inicial para o usuário Colaborador 38
Figura 6 – Tela de execução manual da automação Consulta DTE 39
Figura 7 – Fluxo de execução da aplicação Bot Runner 44
Figura 8 – Tela inicial do portal da UVT antes do login 52
Figura 9 – Tela inicial do portal da UVT após o login 53
Figura 10 – Modal de seleção de empresas 54
Figura 11 – Modal de status do DTE 55

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais 27
Quadro 2 – Requisitos não funcionais 28
Quadro 3 – Resultados observados 57

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
CNPJ	Cadastro Nacional da Pessoa Jurídica
CPF	Cadastro de Pessoa Física
CRUD	Create, Read, Update, Delete
DRY	Don't Repeat Yourself
DTE	Domicílio Tributário Eletrônico
DTO	Data Transfer Objects
GCP	Google Cloud Platform
HTTP	Hypertext Transfer Protocol
IA	Inteligência Artificial
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
RAM	Random-Access Memory
REST	Representational State Transfer
RF	Requisito Funcional
RN	Rio Grande do Norte
RNF	Requisito Não Funcional
ROI	Return on Investment (Retorno sobre o Investimento)
RPA	Robotic Process Automation (Automação Robótica de Processos)
SEFIP	Sistema Empresa de Recolhimento do FGTS e Informações à Previdência Social
SIAP	Sistema Integrado de Automação de Processos
SIGAA	Sistema Integrado de Gestão de Atividades Acadêmicas
TCC	Trabalho de Conclusão de Curso
UFRN	Universidade Federal do Rio Grande do Norte
UVT	Unidade Virtual de Tributação
vCPU	Virtual Central Processing Unit

SUMÁRIO

1 Introdução 13
1.1 Justificativa 14
1.2 Objetivos 14
1.2.1 Objetivo Geral 14
1.2.2 Objetivos Específicos 14
1.3 Estrutura do Trabalho 15
2 Fundamentação Teórica 16
2.1 RPA e Automação de Processos 16
2.2 Tecnologias Utilizadas no Desenvolvimento 17
2.2.1 Linguagem Python e Frameworks 17
2.2.2 Arquitetura de Comunicação REST 18
2.2.3 Conteinerização com Docker 18
2.2.4 Infraestrutura em Nuvem: Google Cloud Platform (GCP) 18
2.2.4.1 Cloud Run 19
2.2.4.2 Cloud SQL 19
2.2.4.3 Pub/Sub 19
2.3 Práticas de Engenharia de Software Moderna 19
2.3.1 Versionamento de Código com Git 20
2.3.2 Logging e Monitoramento 20
2.3.3 Filas de Mensagens e Mecanismos de Retry 20
2.4 Trabalhos Relacionados 21
3 Metodologia 23
3.1 Levantamento e Documentação de Requisitos 23
3.1.1 Identificação dos Atores 23
3.1.2 Funcionalidades como Histórias de Usuário 24
3.1.2.1 Gerenciamento de Usuários e Permissões - Cadastro de Usuários 24
3.1.2.2 Gerenciamento de Clientes - Cadastro de Cliente 25
3.1.2.3 Gerenciamento de Automações - Cadastro de Automação 25
3.1.2.4 Histórico e Logs de Execuções - Consulta de Histórico de Execuções 26
3.1.2.5 Desenvolvimento e Integração de Automações - Execução da Automação "Consulta DTE - UVT 26

3.1.3 Requisitos Funcionais e Não Funcionais 27
3.2 Arquitetura da Solução e Seus Componentes 30
3.2.1 Aplicação Web Django 31
3.2.1.1 Modelos e Responsabilidades 31
3.2.1.2 Fluxo de Solicitação Manual de Execução 35
3.2.2 Bot Runner FastAPI 43
3.2.2.1 Fluxo de Execução 44
3.2.3 Implementação da Automação Piloto: Consulta ao DTE na UVT 50
3.2.3.1 Fluxo do Processo 51
3.2.3.2 Implementação do Bot Consulta DTE 51
4 Resultados Obtidos 57
5 Conclusões 60
Referências 63

1 Introdução

A crescente digitalização de processos e o avanço de tecnologias emergentes vêm transformando significativamente o modo de operação das empresas, em especial no setor contábil. Escritórios de pequeno porte enfrentam o desafio de manter a qualidade dos serviços em um cenário de crescimento acelerado da base de clientes, o que resulta na sobrecarga de colaboradores e na dificuldade de escalar a operação de forma sustentável. Diante disso, a automação de processos por meio de tecnologias como a automação robótica de processos tem se apresentado como uma solução viável.

Conforme aponta Carvalho (2020), a adoção de RPA em escritórios contábeis permite otimizar o tempo dos colaboradores, que passam a ser alocados em tarefas mais estratégicas e analíticas, aumentando a produtividade e a qualidade dos serviços prestados. Entretanto, apesar de seus benefícios, a implementação de RPA também impõe desafios consideráveis. Reis e Faria (2024) destacam que pequenas e médias empresas enfrentam dificuldades específicas, como ausência de suporte técnico qualificado e carência de metodologias adequadas para transformar seus processos. Já Cabral (2022) aponta que, apesar do potencial da tecnologia, há limitações quanto à sua aplicabilidade em rotinas altamente reguladas ou com variação frequente, como as tributárias no contexto brasileiro.

Outro ponto crítico refere-se à baixa resiliência das rotinas automatizadas frente a alterações nos sistemas-alvo, além da escassez de mecanismos eficazes de monitoramento e manutenção. A ausência de governança técnica sobre os robôs pode comprometer a confiabilidade das entregas automatizadas, especialmente quando não há versionamento nem registro adequado de logs (ITREX GROUP, 2023).

Além dos desafios técnicos, há também barreiras relacionadas à usabilidade. A literatura mostra que, quando as ferramentas de automação apresentam interfaces pouco acessíveis ou exigem conhecimento técnico elevado, sua adoção pelos colaboradores tende a ser baixa (PLANEKS, 2023). A familiaridade com os sistemas e a clareza nas funcionalidades oferecidas são fatores-chave para garantir o engajamento dos usuários, sobretudo em ambientes onde o domínio de tecnologias digitais ainda está em desenvolvimento.

1.1 Justificativa

De acordo com Cabral (2022), a adoção de RPA pode efetivamente ampliar o perfil gerencial dos contadores e torná-los mais produtivos, à medida que libera tempo para a atuação em atividades de maior valor agregado. Adicionalmente, enquanto grandes empresas contam com infraestruturas robustas para implementação de soluções desse tipo, os pequenos negócios frequentemente não dispõem de alternativas adaptadas à sua realidade operacional (REIS e FARIA, 2024; GARTNER, 2023). Dessa forma, a relevância deste trabalho reside na proposta de desenvolver uma plataforma web que incorpore boas práticas de engenharia de software e não se limite a executar scripts de automação, mas que também forneça funcionalidades para o gerenciamento centralizado, monitoramento da execução e aumento da resiliência desses scripts. Tal plataforma visa endereçar as lacunas frequentemente observadas em implementações de RPA, como a dificuldade de manutenção e a falta de visibilidade sobre o desempenho das automações.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver uma plataforma web para gerenciamento e execução de scripts de automação de processos contábeis, adaptada ao contexto de um escritório contábil de pequeno porte em expansão.

1.2.2 Objetivos Específicos

- Planejar e definir o escopo mínimo viável (MVP) da plataforma com base em requisitos reais;
- Implementar uma arquitetura modular e extensível que facilite a manutenção das automações;
- Automatizar pelo menos um processo contábil de alto impacto no cotidiano do escritório;
- Incorporar mecanismos de rastreamento, logs e monitoramento de execuções;
- Avaliar a usabilidade e robustez da plataforma com base em testes técnicos e feedback do usuário final.

1.3 Estrutura do Trabalho

Este trabalho está organizado em cinco capítulos. O Capítulo 1 apresenta a introdução do projeto, com a contextualização do problema, a justificativa, os objetivos propostos e uma visão geral da estrutura do documento. O Capítulo 2 é dedicado à fundamentação teórica, abordando os conceitos de RPA e automação de processos, práticas modernas de engenharia de software, arquiteturas voltadas à modularidade e extensibilidade, bem como trabalhos relacionados que servem de base para a proposta desenvolvida. O Capítulo 3 descreve a metodologia adotada para a realização do projeto, com foco no planejamento, especificação e desenvolvimento da solução. O Capítulo 4 apresenta os resultados obtidos, incluindo a demonstração do MVP funcional, a avaliação técnica da solução implementada e o feedback fornecido pelo escritório parceiro. Por fim, o Capítulo 5 apresenta as conclusões do trabalho, discute suas limitações e propõe direções para pesquisas e melhorias futuras.

2 Fundamentação Teórica

Este capítulo apresenta os fundamentos teóricos e tecnológicos que servem como alicerce para o desenvolvimento deste trabalho. A abordagem inicia-se com a definição de RPA e seu papel no contexto contábil, detalhando as ferramentas escolhidas para sua implementação. Em seguida, são explorados conceitos da Engenharia de Software Moderna, cujas práticas são essenciais para construir uma solução robusta e manutenível. Posteriormente, são discutidas arquiteturas de sistemas distribuídos, fundamentais para a compreensão do design da plataforma. Por fim, é realizada uma análise de trabalhos relacionados para situar esta pesquisa no panorama tecnológico e acadêmico atual.

2.1 RPA e Automação de Processos

RPA, ou Automação Robótica de Processos, pode ser definida como uma aplicação tecnológica que busca automatizar, por meio de robôs de software, tarefas demasiadamente repetitivas (CARVALHO, 2020). Também pode ser descrita como uma força de trabalho digital que pode ser configurada para executar tarefas baseadas em regras através da interação com os sistemas da mesma forma que um usuário humano faria (DELOITTE, 2022). Os sistemas de RPA operam na camada de apresentação (a interface gráfica), interagindo com os softwares e aplicações automatizadas de forma programática, simulando ações humanas, o que permite a implementação em ambientes com sistemas legados ou de difícil integração, que não disponibilizam APIs para interação.

No setor contábil, a aplicação de RPA tem se mostrado particularmente eficaz. Processos como consultas de informações em portais, download de arquivos e relatórios, lançamentos fiscais, geração de guias de recolhimento de tributos e muitos outros exemplos são, em grande parte, repetitivos e baseados em regras, tornando-se excelentes candidatos para a automação. A implementação de RPA nestas áreas pode levar a benefícios significativos, incluindo a redução de erros operacionais, aumento da produtividade e a liberação de profissionais para atividades de maior valor agregado, como consultoria estratégica (GARTNER, 2023).

2.2 Tecnologias Utilizadas no Desenvolvimento

A escolha das tecnologias foi um pilar fundamental para viabilizar a construção da aplicação proposta. A seguir, são detalhadas as principais ferramentas e serviços que compõem a solução.

2.2.1 Linguagem Python e Frameworks

Python é uma linguagem de programação de alto nível, interpretada e multiparadigma cuja filosofia de design enfatiza a legibilidade do código com uma sintaxe que permite aos programadores expressar conceitos em menos linhas do que seria possível em outras linguagens (PYTHON.ORG, 2025). Seu vasto ecossistema de bibliotecas e *frameworks* a torna uma ferramenta extremamente poderosa e flexível tanto para o desenvolvimento de automações, quanto para aplicações Web, exatamente os dois tipos de sistema abarcados por esse projeto. Por esse motivo, a escolha do Python como linguagem principal é completamente justificável. Dentro desse ecossistema, as seguintes ferramentas se destacam e foram amplamente utilizadas:

- **Django:** framework web de alto nível que incentiva o desenvolvimento rápido e o design limpo e pragmático. Ele segue o princípio *"Don't Repeat Yourself"* (DRY) e vem com uma filosofia de "baterias inclusas", oferecendo uma vasta gama de funcionalidades prontas como um sistema de autenticação e autorização robusto e um painel administrativo (Django Admin) totalmente funcional e extensível (DJANGO, 2025). Neste projeto, o Django foi utilizado para construir a aplicação Web principal, responsável pela gestão de usuários, clientes, automações e pelo painel de controle.

- **FastAPI:** framework web moderno, de alta performance, leve e minimalista, ideal para construção de REST APIs (FASTAPI, 2025). No projeto, o FastAPI foi utilizado para implementar os Bot Runners, serviços leves e eficientes cuja única responsabilidade é receber uma tarefa, executar a automação e comunicar o resultado à aplicação Web principal através de uma requisição HTTP.

- **Selenium:** framework de código aberto para automação de navegadores web. Ele fornece um conjunto de ferramentas e bibliotecas que permitem aos desenvolvedores escrever scripts que controlam um navegador, simulando as ações de um usuário, como clicar em botões, preencher formulários e navegar entre páginas (SELENIUM, 2025). Para este projeto, o Selenium é a peça-chave que permite aos robôs interagir com os portais e sistemas contábeis baseados na web.

2.2.2 Arquitetura de Comunicação REST

A comunicação entre os diferentes serviços da plataforma (especificamente, entre os *Bot Runners* e a Aplicação Django) segue os princípios da arquitetura REST (Representational State Transfer). REST é um estilo arquitetural que define um conjunto de restrições para a criação de serviços web que utiliza os métodos padrão do protocolo HTTP (GET, POST, PUT, PATCH e DELETE) para operar sobre recursos (FIELDING, 2000). Neste caso, o recurso em questão trata-se de uma execução de automação e as operações que podem ser realizadas sobre ele são leitura (GET), criação (POST) e atualização (PUT/PATCH). Essa abordagem simplifica a comunicação, promove o baixo acoplamento entre os serviços e é amplamente adotada pela indústria, facilitando a integração e a manutenção do sistema.

2.2.3 Conteinerização com Docker

Para garantir que a aplicação e os robôs de automação executem de forma consistente em diferentes ambientes (desenvolvimento, testes, produção), foi utilizada a tecnologia de contêineres Docker. Um contêiner empacota o código da aplicação com todas as suas dependências (bibliotecas, arquivos de configuração, etc.) em uma unidade isolada e portátil, criando um ambiente padronizado para a aplicação, o que permite a reprodução exata do ambiente no qual o sistema foi projetado e simplifica drasticamente o processo de implantação e distribuição (DOCKER, 2025).

2.2.4 Infraestrutura em Nuvem: Google Cloud Platform (GCP)

Toda a infraestrutura da aplicação foi implantada no serviço de computação em nuvem do Google, a Google Cloud Platform (GCP), utilizando um conjunto de soluções gerenciadas que oferecem escalabilidade, confiabilidade e otimização de custos. A seguir será apresentada uma breve descrição de cada uma das soluções utilizadas.

2.2.4.1 Cloud Run

Plataforma de computação *serverless* que permite executar contêineres Docker sob demanda. Ele escala automaticamente o número de instâncias com base no tráfego, podendo escalar até zero quando não há requisições, eliminando custos com ociosidade (GOOGLE CLOUD, 2025). Neste projeto, tanto a Aplicação Django quanto os *Bot Runners* em FastAPI são implantados como serviços no Cloud Run.

2.2.4.2 Cloud SQL

Serviço de banco de dados relacional totalmente gerenciado. Ele automatiza tarefas complexas como backups, replicação, patches e atualizações, permitindo que o desenvolvedor se concentre na aplicação (GOOGLE CLOUD, 2025). Para este projeto, foi utilizada a instância para PostgreSQL.

2.2.4.3 Pub/Sub

Serviço de mensageria assíncrona e em tempo real que permite que os serviços se comuniquem através de um modelo *publish-subscribe*, no qual a Aplicação *publisher* envia uma mensagem (solicitação de execução de automação, por exemplo) para um tópico, que atua como uma espécie de caixa de mensagens, e as aplicações *subscribers* consomem essas mensagens, realizando o processamento necessário (GOOGLE CLOUD, 2025). Esse padrão de comunicação desacopla os componentes e promove maior resiliência, garantindo que o sistema continue funcionando mesmo que uma das partes esteja temporariamente indisponível, já que as mensagens publicadas nos tópicos podem ser consumidas à medida que o sistema se recupere de possíveis problemas ou instabilidades.

2.3 Práticas de Engenharia de Software Moderna

A simples criação de scripts de automação não é suficiente para garantir uma solução de RPA escalável e confiável. Conforme discutido na introdução deste trabalho, iniciativas de RPA frequentemente enfrentam desafios relacionados à manutenção dos robôs, monitoramento de falhas e orquestração (ITREX GROUP, 2023). A aplicação de práticas da Engenharia de Software Moderna é fundamental para mitigar esses problemas e construir uma plataforma robusta. Para Valente (2022), a Engenharia de Software Moderna se preocupa

com o ciclo de vida completo do software, desde a concepção até a implantação e manutenção, com foco em práticas que garantam qualidade e agilidade. A seguir, são detalhadas algumas práticas adotadas neste projeto:

2.3.1 Versionamento de Código com Git

O controle de versão é uma prática essencial no desenvolvimento de qualquer software. Valente (2022) afirma que sistemas de controle de versão são importantes pois permitem que desenvolvedores de um mesmo time trabalhem de forma colaborativa e assíncrona em um mesmo projeto com o mínimo de atrito. Além disso, utilizando o Git, é possível manter um histórico completo de todas as alterações realizadas no código-fonte dos scripts de automação e da plataforma, facilitando a identificação de quando um erro foi introduzido, permitindo reverter para versões anteriores e gerenciando o desenvolvimento de novas funcionalidades em *branches* separadas sem afetar a versão estável.

2.3.2 Logging e Monitoramento

Sistemas de automação, por operarem de forma autônoma e muitas vezes realizarem rotinas críticas, que podem, inclusive, acarretar na geração de danos e prejuízos como multas caso não funcionem corretamente, precisam ser observáveis. A prática de *logging*, ou geração de registros de eventos, crucial para o monitoramento e a depuração, consiste em registrar informações relevantes durante a execução do software, como o início e o fim de um processo ou erros inesperados. Esses registros são a principal ferramenta para que os desenvolvedores e a equipe de suporte possam diagnosticar problemas após sua ocorrência (VALENTE, 2022).

2.3.3 Filas de Mensagens e Mecanismos de Retry

Para orquestrar a execução dos scripts de forma escalável e resiliente, a implementação de uma fila de mensagens é uma abordagem padrão em sistemas distribuídos. Quando uma automação é solicitada, em vez de ser executada imediatamente pela aplicação principal, uma mensagem descrevendo a tarefa é publicada em uma fila. Conforme Richardson (2018), em seu livro sobre padrões de microsserviços, a comunicação assíncrona baseada em mensagens promove um baixo acoplamento, permitindo que o produtor da mensagem (a aplicação Django) e o consumidor (o *Bot Runner*) operem e escalem de forma independente.

Além disso, falhas transitórias, como instabilidade de rede ou indisponibilidade momentânea de um serviço Web, são comuns. Então, para mitigar esse problema, o sistema de mensageria utilizado no projeto, o Google Cloud Pub/Sub, possui mecanismos de *retry* (repetição) e confirmação de recebimento (*acknowledgment*), de modo que, se um *Bot Runner* falha ao processar uma tarefa ou o recebimento da mensagem não é confirmado, ela será entregue novamente para outra tentativa, aumentando consideravelmente a resiliência do sistema.

2.4 Trabalhos Relacionados

A aplicação de RPA em contabilidade e o desenvolvimento de plataformas de orquestração são temas de interesse tanto na indústria quanto na academia, conforme demonstrado pela existência de trabalhos anteriores sobre o assunto.

O trabalho de Carvalho (2020) é uma referência próxima, focando na automação de um serviço contábil específico, o SEFIP - Conectividade Social, utilizando Python e suas bibliotecas. O estudo detalha a construção de um robô para uma tarefa pontual, demonstrando ganhos de produtividade. O presente trabalho, embora utilize tecnologias similares, diferencia-se por propor o desenvolvimento de uma plataforma de gerenciamento genérica e extensível, capaz de orquestrar múltiplos robôs distintos, em vez de focar em uma única automação. Dessa forma, este trabalho apresenta-se de forma complementar ao que estudo supracitado, resolvendo dores provenientes da implementação de automações RPA, como melhorias na usabilidade, capacidade de monitoramento e aplicação de técnicas para aumento de resiliência.

Em uma revisão sistemática da literatura, Reis e Faria (2024) investigaram o desenvolvimento do profissional contábil diante da RPA. Eles concluíram que a tecnologia não visa substituir o contador, mas sim modificar seu perfil, exigindo novas competências e realocando o tempo de tarefas repetitivas para atividades mais analíticas e estratégicas. Esse achado reforça a justificativa deste projeto, que busca fornecer uma ferramenta que permita a implementação sustentável de aplicações de RPA de modo a liberar os colaboradores para realização de trabalho de maior valor.

Vale a pena também salientar que existem grandes soluções comerciais no mercado como *UiPath*, *Automation Anywhere* e *Blue Prism* que oferecem plataformas de orquestração

de RPA extremamente robustas, com funcionalidades avançadas de agendamento, monitoramento, governança e até mesmo IA, mas os custos elevados e a falta de flexibilidade para lidar com processos nem sempre bem estruturados podem ser proibitivos para escritórios de contabilidade de pequeno e médio porte, público alvo a quem este projeto se destina inicialmente. Para isso, busca-se explorar uma arquitetura mais enxuta e customizável, utilizando tecnologias de código aberto e serviços de nuvem de baixo custo inicial, oferecendo uma alternativa viável para este nicho.

Portanto, este trabalho se posiciona na interseção da aplicação prática de RPA no contexto contábil com a aplicação de princípios de engenharia de software para construir uma plataforma de orquestração acessível, modular e escalável.

3 Metodologia

Neste capítulo, será apresentado o processo de desenvolvimento adotado para implementação do projeto, começando com a descrição do processo de levantamento dos requisitos, de suma importância para concepção de qualquer software, passando por uma visão geral da arquitetura adotada, que permitirá a compreensão de quais elementos compõem a aplicação e como eles se comunicam, e chegando ao detalhamento, de forma sucinta, porém abrangente, de cada um dos componentes presentes na arquitetura descrita. Por fim, o processo escolhido como automação piloto, a Consulta ao Domicílio Tributário Eletrônico (DTE) no portal da Unidade Virtual de Tributação (UVT) do Estado do Rio Grande do Norte (RN), será apresentado, juntamente com a justificativa do motivo pelo qual foi escolhido e os resultados obtidos com a partir da sua automatização.

3.1 Levantamento e Documentação de Requisitos

Para a especificação do sistema proposto, realizou-se um levantamento e documentação dos requisitos em três etapas principais, detalhadas a seguir: identificação dos atores, descrição das funcionalidades através de Histórias de Usuário e, por fim, a listagem dos Requisitos Funcionais e Não Funcionais. Esta abordagem visa garantir uma compreensão clara do escopo do MVP e das características da solução definida. Nos tópicos a seguir serão apresentados, de forma sucinta, os principais resultados obtidos no processo de levantamento de requisitos.

3.1.1 Identificação dos Atores

Com base no escopo do MVP, especialmente no que se refere ao gerenciamento de usuários e suas respectivas permissões, foram identificados os seguintes perfis que utilizarão o sistema, aqui denominados atores:

- **Gestor:** Representa o usuário do escritório parceiro com responsabilidades administrativas sobre clientes e automações. Este ator também supervisiona as operações através de consultas de históricos e logs gerados pelo sistema e pode executar automações.

- **Colaborador:** Usuário do escritório parceiro focado na execução de automações para os clientes e na consulta de históricos e logs básicos gerados por essas execuções.
- **Desenvolvimento/Suporte:** Perfil técnico referente a equipe de operações e suporte responsável por manter a saúde e correto funcionamento da solução. Este ator possui acesso irrestrito ao sistema para manutenções, evoluções, configurações avançadas e resolução de problemas críticos.

3.1.2 Funcionalidades como Histórias de Usuário

As funcionalidades planejadas para o MVP foram descritas na forma de Histórias de Usuário e agrupadas em Épicos, ambos amplamente utilizados em frameworks de gerenciamento ágil de projetos como o *Scrum*. As Histórias de Usuário buscam expressar cada funcionalidade sob a perspectiva do ator que a utilizará, o que ele deseja realizar e o valor esperado com essa ação. Já os Épicos são coleções de histórias que descrevem uma funcionalidade ou parte maior do sistema. Os tópicos a seguir descrevem os Épicos nos quais os requisitos foram divididos junto com suas principais histórias.

3.1.2.1 Gerenciamento de Usuários e Permissões - Cadastro de Usuários

Como Desenvolvimento/Suporte, quero cadastrar usuários e atribuí-los aos seus respectivos grupos (Gestores, Colaboradores, Desenvolvimento/Suporte) com permissões específicas no *Django Admin*, para que o acesso às funcionalidades e dados do sistema seja controlado de forma segura e adequada a cada papel.

Critérios de aceitação:

- Os usuários devem possuir os campos “usuário”, “senha” como obrigatórios e o campo “email” como opcional.
- Devem existir três grupos: "Gestores", "Colaboradores", "Desenvolvimento/Suporte".
- O grupo "Gestores" deve ter permissão para gerenciar Clientes e Automações, visualizar Histórico e Logs.
- O grupo "Colaboradores" deve ter permissão para visualizar Clientes, executar Automações e consultar Histórico e Logs.
- O grupo "Desenvolvimento/Suporte" deve ter acesso administrativo completo.

3.1.2.2 Gerenciamento de Clientes - Cadastro de Cliente

Como Gestor, quero cadastrar os dados de um novo cliente (Nome/Razão Social, CNPJ/CPF, Email, Status), para que ele possa ser selecionado em execuções de automações e gerenciado pela plataforma.

Critérios de aceitação:

- Os campos Nome/Razão Social e CNPJ/CPF devem ser obrigatórios.
- O campo CNPJ/CPF deve ter um formato válido (validação básica).
- O campo Email deve ter um formato válido.
- O Status do cliente (ativo/inativo) deve ser definido no cadastro, sendo "ativo" o padrão.
- O cliente cadastrado deve aparecer na lista de clientes.

3.1.2.3 Gerenciamento de Automações - Cadastro de Automação

Como Desenvolvimento/Suporte, quero cadastrar uma nova automação (Nome, Descrição curta, Script/módulo associado, Status, Clientes associados), para que ela fique disponível para execução na plataforma.

Critérios de aceitação:

- Os campos Nome e Script/módulo associado devem ser obrigatórios.
- O Status da automação (ativa/inativa) deve ser definido no cadastro, sendo "ativa" o padrão.
- Deve ser possível associar a automação a um ou mais clientes existentes.
- A automação cadastrada deve aparecer na lista de automações.

3.1.2.4 Histórico e Logs de Execuções - Consulta de Histórico de Execuções

Como Colaborador ou Gestor, quero consultar o histórico de execuções das automações, podendo filtrar por cliente, automação ou período, para que possa auditar e verificar os resultados das automações realizadas.

Critérios de aceitação:

- O histórico deve exibir: Cliente(s) envolvido(s), Automação executada, Data/hora de início, Data/hora de fim (se aplicável), e Status final da execução.
- Deve ser possível acessar os logs básicos de uma execução a partir do histórico.

3.1.2.5 Desenvolvimento e Integração de Automações - Execução da Automação "Consulta DTE - UVT"

Como Colaborador ou Gestor, quero executar a automação "Consulta DTE - UVT" para os clientes selecionados, para que seja possível verificar se existem mensagens não lidas no Domicílio Tributário Eletrônico da Unidade Virtual de Tributação.

Critérios de aceitação:

- A automação "Consulta DTE - UVT" deve estar listada como uma automação disponível para seleção (se ativa).
- A execução deve seguir o fluxo padrão de seleção de cliente(s), disparo e acompanhamento.
- Os resultados específicos ou status da consulta DTE devem ser refletidos nos logs ou em um campo de resultado da execução.

3.1.3 Requisitos Funcionais e Não Funcionais

A partir do escopo do MVP e das Histórias de Usuário, foram derivados diversos requisitos funcionais, os quais descrevem o que o sistema deve fazer, e requisitos não funcionais, que especificam como o sistema deve operar. A tabela a seguir descreve o principal requisito funcional de cada Épico.

ÉPICO	REQUISITO FUNCIONAL
Gerenciamento de Usuários e Permissões	**RF001:** O sistema deve permitir ao perfil de Desenvolvimento/Suporte criar e gerenciar grupos de usuários (Gestores, Colaboradores, Desenvolvimento/Suporte) com permissões distintas, utilizando a interface administrativa do Django.
Gerenciamento de Clientes	**RF002:** O sistema deve permitir que os Gestores realizem o cadastro, visualização, edição e listagem de Clientes. O cadastro de Cliente deve contemplar os campos: Razão Social (obrigatório), CNPJ/CPF (obrigatório, com validação de formato básico), E-mail (com validação de formato) e Status (ativo/inativo sendo ativo o valor padrão).
Gerenciamento de Automações	**RF003:** O sistema deve permitir ao perfil de Desenvolvimento/Suporte realizar o cadastro, visualização, edição e listagem de Automações. O cadastro de Automação deve contemplar os campos: Nome (obrigatório), Descrição, URL de Execução (*endpoint* do *Bot Runner* responsável por executar a automação) e Status (ativo/inativo sendo ativo o valor padrão).

Execução de Automações	**RF004:** O sistema deve possibilitar que Colaboradores e Gestores selecionem um ou mais Clientes ativos na tela de execução de uma determinada Automação ativa para enviar, de forma manual, uma solicitação de execução ao *Bot Runner*.
Histórico e Logs de Execuções	**RF005:** O sistema deve manter um histórico de todas as execuções de automação, registrando no mínimo: Cliente(s) envolvido(s), Automação executada, Data/hora de início, Data/hora de fim (quando aplicável), status final e o usuário que solicitou a execução.

Quadro 1 - Requisitos funcionais

A próxima tabela descreve alguns dos principais requisitos não funcionais divididos em categorias pertinentes para este projeto.

CATEGORIA	REQUISITO NÃO FUNCIONAL
Arquitetura e Tecnologia	**RNF001:** A aplicação web principal será desenvolvida com o framework *Django* e os *Bot Runners*, responsáveis pela execução das automações, utilizarão o framework *FastAPI* e a biblioteca *Selenium*, ambos com a linguagem de programação Python, e devem ser implantados como serviços no *Google Cloud Run*, comunicando-se via solução de mensageria assíncrona *Google Cloud Pub/Sub.*
Segurança	**RNF002:** O acesso às funcionalidades do sistema será controlado por autenticação e autorização baseadas nos grupos de permissão (Gestores, Colaboradores, Desenvolvimento/Suporte), utilizando os recursos nativos do *Django*.

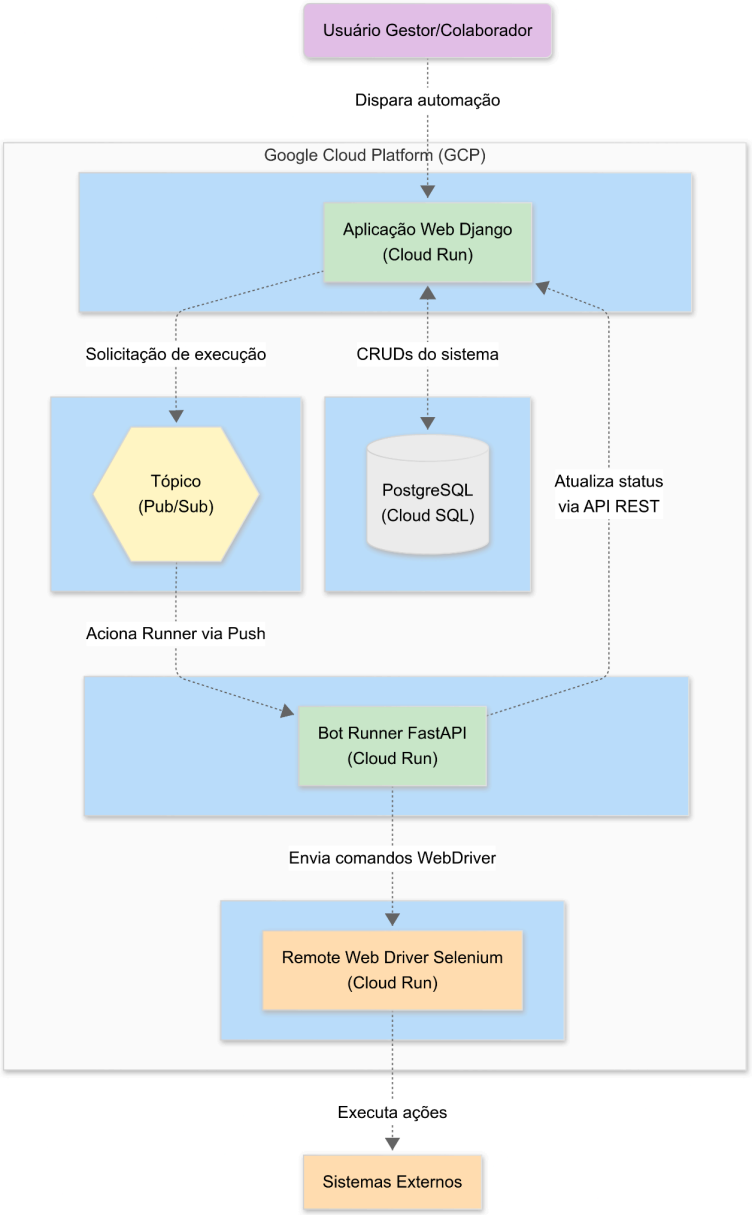
Escalabilidade	**RNF003:** A arquitetura deverá suportar a escalabilidade dos *Bot Runners* (múltiplas instâncias no *Cloud Run*) para lidar com um volume crescente de execuções concorrentes, considerando as configurações do *Pub/Sub* e a capacidade do banco de dados.
Confiabilidade	**RNF04:** O sistema deve incluir tratamento de exceções durante a execução das automações, reportando o status de erro de forma apropriada e ser capaz de executar novamente (*retry*) quando apropriado.

Quadro 2 - Requisitos não funcionais

3.2 Arquitetura da Solução e Seus Componentes

A arquitetura da solução foi concebida com base em um modelo de sistema distribuído, modular e orientado a serviços, implantado integralmente na plataforma de computação em nuvem do Google, a *Google Cloud Platform* (GCP). Essa abordagem foi escolhida para atender a requisitos essenciais do projeto, como escalabilidade, resiliência e facilidade de manutenção. A imagem a seguir ilustra os componentes dessa arquitetura e sua comunicação:

Figura 1 - Arquitetura da solução



Fonte - Elaborado pelo autor (2025)

O design ilustrado na Figura 1 separa claramente as responsabilidades entre a gestão da plataforma, a mensageria assíncrona e a execução das automações. A comunicação entre os principais componentes, a aplicação *Django* e o *Bot Runner FastAPI*, é realizada de forma desacoplada, garantindo que falhas em uma das partes tenham impacto mínimo sobre a outra. A seguir, serão descritos os componentes principais dessa arquitetura, cujas interações foram ilustradas no diagrama apresentado anteriormente. Partes da implementação serão omitidas pela inviabilidade de se expôr na íntegra um projeto que, apesar de ainda estar em fase embrionária e se tratar de um MVP, já possui volume de código considerável. Além disso, este trabalho não tem por objetivo servir de documentação técnica, portanto, entende-se que não guarda obrigação de descer a tal nível de especificidade.

3.2.1 Aplicação Web Django

Componente que constitui o núcleo do sistema de automação de processos contábeis, sendo responsável pelo gerenciamento centralizado de usuários, clientes, automações e execuções. Implementada com a linguagem Python e o framework Django, esta aplicação fornece uma interface administrativa robusta para os operadores do escritório contábil interagirem e foi projetada para operar de forma escalável, integrando-se a outros serviços via mensageria assíncrona utilizando o padrão *publish-subscribe* (publicação-assinatura, em tradução literal) através do serviço *Pub/Sub* da plataforma de computação em nuvem do Google, a *Google Cloud Platform* (GCP). Na arquitetura desenvolvida, esta aplicação desempenha o papel de *publisher*, publicando as solicitações de execução que serão consumidas por outras aplicações, os *Bot Runners*, que serão detalhadas em tópicos posteriores deste mesmo trabalho.

3.2.1.1 Modelos e Responsabilidades

A aplicação *Django* é executada em um ambiente conteinerizado (Docker), hospedada no *Cloud Run*, e utiliza o *Cloud SQL* (*PostgreSQL*) como banco de dados relacional, ambos serviços também da GCP. Como citado anteriormente, sua principal responsabilidade é fornecer funcionalidades de CRUD (*Create*, *Read*, *Update*, *Delete*) para entidades centrais do domínio – Clientes, Automações e Execuções – além de gerenciar usuários e permissões por meio dos modelos padrão de usuário e grupos fornecidos pelo *Django*. A seguir, serão apresentados trechos de código com a definição básica das entidades citadas anteriormente, chamadas de *models* (modelos) no ecossistema *Django* – classes especiais que implementam

métodos de interação com o banco de dados e realizam operações de leitura e escrita através de uma interface simplificada fornecida por um sistema de ORM (Object-relational mapping, ou Mapeamento objeto-relacional em tradução literal) padrão embutido no próprio framework. Para simplificar a apresentação das entidades, alguns métodos de validação e outros auxiliares serão omitidos.

O primeiro modelo apresentado logo abaixo, o *Client*, é responsável por armazenar os dados dos clientes para os quais as automações serão executadas.

```
class Client(WithExternalUUID):
    company_name = models.CharField("Razão Social", max_length=255)
    email = models.EmailField("E-mail")
    cnpj = models.CharField(
        "CNPJ",
         max_length=14,
         blank=True,
         null=True,
         unique=True,
    )
    cpf = models.CharField(
        "CPF",
         max_length=11,
         blank=True,
         null=True,
         unique=True,
    )
    active = models.BooleanField("Ativo", default=True)
    automations = models.ManyToManyField(Automation, related_name="clients")

    def clean(self):
       cnpj_filled = bool(self.cnpj)
       cpf_filled = bool(self.cpf)

       if cnpj_filled and cpf_filled:
           raise ValidationError("Preencha apenas CNPJ ou CPF, nunca ambos.")

       if not cnpj_filled and not cpf_filled:
           raise ValidationError("Preencha CNPJ ou CPF.")

       if self.cnpj and (not self.cnpj.isdigit() or len(self.cnpj) != 14):
           raise ValidationError("CNPJ deve conter 14 dígitos numéricos.")

       if self.cpf and (not self.cpf.isdigit() or len(self.cpf) != 11):
           raise ValidationError("CPF deve conter 11 dígitos numéricos.")
```

Os clientes podem ser pessoas físicas ou jurídicas. Por esse motivo, o modelo possui os campos CPF (Cadastro de Pessoa Física) e CNPJ (Cadastro Nacional da Pessoa Jurídica), mas somente um deles poderá ser preenchido para uma mesma instância. A implementação dessa validação acontece no método *clean*. Outro ponto a se observar é a implementação, de uma forma simples e objetiva, graças ao framework, da relação muitos-para-muitos através

do campo *automations*, do tipo *ManyToManyField*. Com isso, uma instância de cliente se relaciona com várias instâncias de automação, e vice-versa.

O modelo *Automation*, que será apresentado no trecho a seguir, é responsável por armazenar os dados da uma automação como o processo ao qual se refere e o tópico do sistema de mensageria responsável por armazenar as solicitações de execução para a respectiva automação.

```
class Automation(WithExternalUUID, WithUniqueSlug):
   class ProcessChoices(models.TextChoices):
       DEBUG = "DEBUG", "Debug"
       CONSULTA_DTE = "CONSULTA_DTE", "Consulta DTE"

   application = models.ForeignKey(
       Application,
       on_delete=models.PROTECT,
       verbose_name="Aplicação",
       related_name="automations"
   )
   process = models.CharField(
       "Processo",
       max_length=64,
       choices=ProcessChoices
   )
   description = models.CharField(
       "Descrição",
       max_length=255,
       blank=True, null=True
   )
   topic_name = models.CharField(
       "Nome do Tópico (Pub/Sub)",
       max_length=255,
       default=settings.GCP_PUBSUB_DEFAULT_TOPIC,
   )
   active = models.BooleanField("Ativa", default=True)
```

Assim como no modelo apresentado anteriormente, o *Client*, nesse também percebe-se a presença de um relacionamento. Porém, aqui, na cardinalidade um-para-muitos com o modelo *Application* – entidade auxiliar responsável por armazenar dados específicos de uma determinada aplicação que será automatizada, como login e senha, por exemplo. Esse relacionamento acontece através do campo *application*, do tipo *ForeignKey* (chaves estrangeiras).

Para viabilizar a execução das automações, foi necessário estabelecer um modelo de dados que materializasse a associação entre uma automação específica e um cliente. Essa escolha de design se baseou no funcionamento do painel administrativo do Django, que, por padrão, exibe uma tela de listagem para cada modelo concreto registrado, em que cada linha representa uma instância. Os modelos de execução foram concebidos com este

comportamento em mente: cada instância representa um par automação-cliente pronto para ser acionado. A principal vantagem dessa abordagem é a capacidade de adicionar ações personalizadas a essa tela de listagem, permitindo que um usuário selecione uma ou mais dessas associações (instâncias de modelo execução) e dispare, com um único clique, o fluxo que publica as solicitações de execução no *Pub/Sub*. O trecho de código a seguir ilustra os campos e relacionamentos descritos acima:

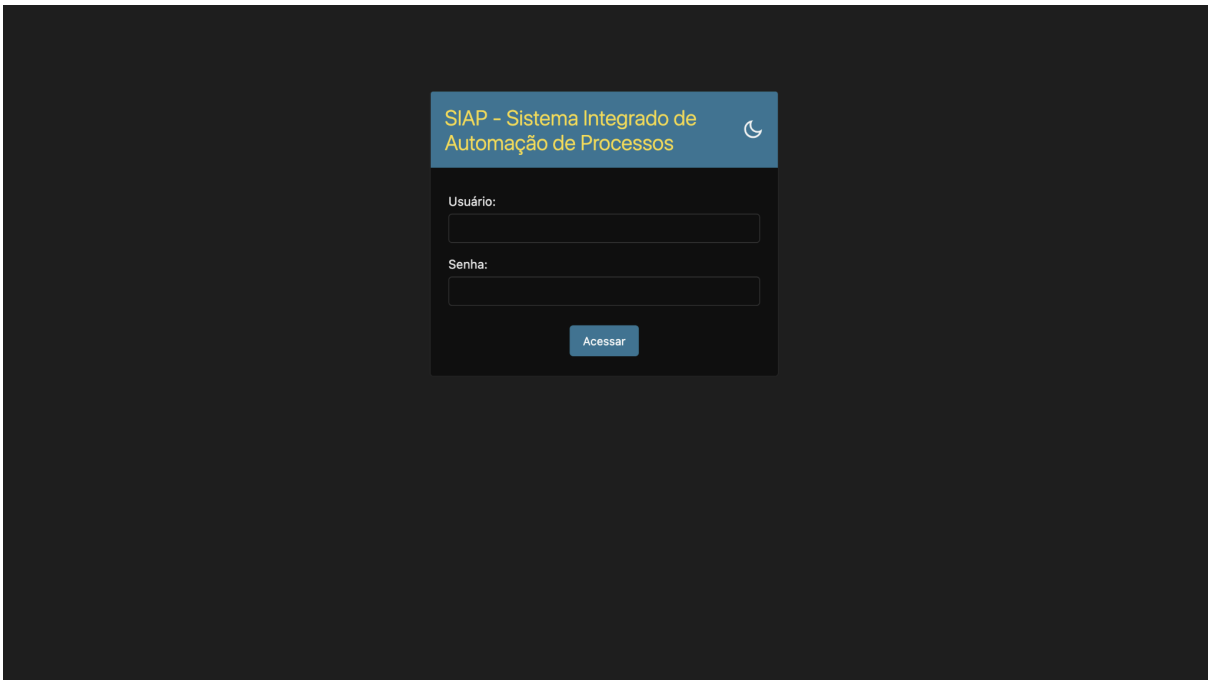
```
class BaseExecution(WithExternalUUID, WithTimeStamp):

   class StateChoices(models.TextChoices):
       PENDING_STATE = "PEND", "Pendente"
       IN_EXECUTION_STATE = "IN_EXEC", "Em Execução"
       DONE_STATE = "DONE", "Concluída"
       ERROR_STATE = "ERROR", "Erro"
       __empty__ = "-"

   automation = models.ForeignKey(
       automation.Automation,
       on_delete=models.PROTECT,
       editable=False,
       verbose_name="Automação",
   )
   client = models.ForeignKey(
       client.Client,
       on_delete=models.PROTECT,
       editable=False,
       verbose_name="Cliente",
   )
   current_state = models.CharField(
       "Estado Atual",
       max_length=15,
       choices=StateChoices,
       blank=True,
       null=True,
       editable=False,
   )
   last_state = models.CharField(
       "Último Estado",
       max_length=15,
       choices=StateChoices,
       blank=True,
       null=True,
       editable=False,
   )

   class Meta:
       abstract = True


class ConsultaDTEExecution(BaseExecution):
   unread_messages = models.SmallIntegerField(
       "Mensagens não lidas", blank=True, null=True
   )
   read_messages = models.SmallIntegerField(
       "Mensagens lidas", blank=True, null=True
   )
```

A fim de evitar a duplicação de código e garantir um comportamento padrão para todos os tipos de execução, a estrutura foi implementada utilizando uma classe base abstrata, a *BaseExecution*. No Django, um modelo abstrato (*Meta: abstract = True*) funciona como um molde, definindo campos e métodos comuns que são herdados por outros modelos, mas sem criar uma tabela própria no banco de dados. Dessa forma, modelos concretos – como *ConsultaDTEExecution* – herdam toda a lógica de associação entre *Client* e *Automation* e os campos de gerenciamento de estado, ao mesmo tempo em que possuem a flexibilidade de adicionar campos próprios para armazenar resultados específicos de sua respectiva tarefa, como é o caso dos contadores de mensagens no *ConsultaDTEExecution*.

3.2.1.2 Fluxo de Solicitação Manual de Execução

Ao acessar o sistema, assumindo que o usuário não esteja logado, ele poderá visualizar uma tela simples de login, com um formulário contendo os campos usuário e senha, conforme a figura abaixo:

Figura 2 - Tela de login



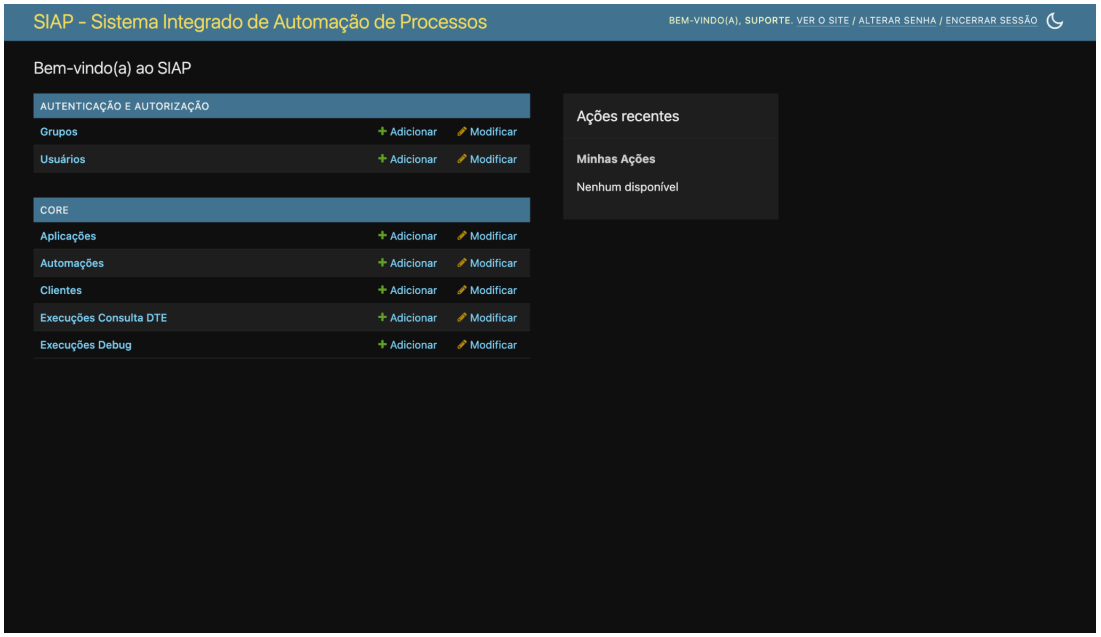
Fonte - Elaborado pelo autor (2025)

Após realizar o login preenchendo o formulário exibido na Figura 2, o usuário será direcionado para tela inicial da aplicação, na qual estarão presentes os menus de acesso às

demais funcionalidades. As opções exibidas nesta tela serão diferentes para cada tipo de usuário logado (Desenvolvimento/Suporte, Gestor ou Colabor).

As capturas de tela nas figuras a seguir são as visualizações que cada um dos perfis terá ao realizar login no sistema. Logo depois, explorar-se-á o fluxo de execução manual a partir da perspectiva de um usuário do tipo colaborador, que possui os menores privilégios de acesso, porém suficientes para esta ação.

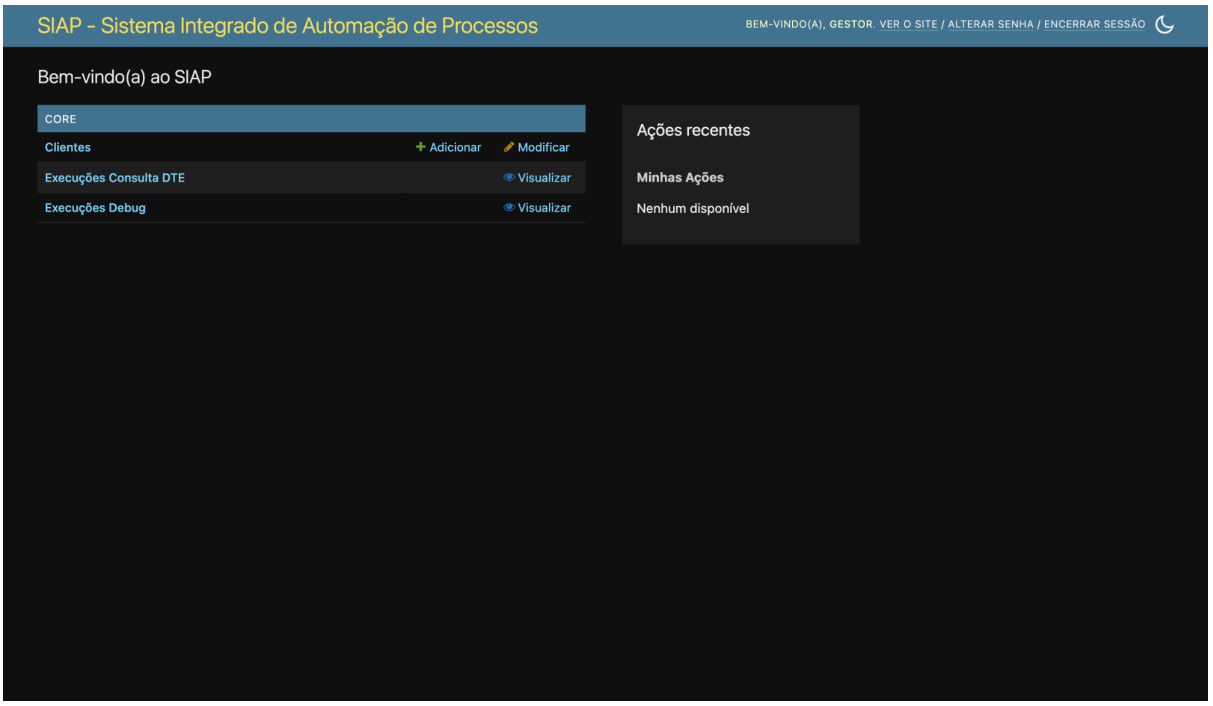
Figura 3 - Tela inicial para o usuário Desenvolvimento/Suporte



Fonte - Elaborado pelo autor (2025)

Usuários do tipo Desenvolvimento/Suporte possuem o acesso mais completo ao sistema, podendo visualizar todas as entidades e executar todas as ações disponíveis, incluindo administrativas como cadastro e edição de usuários e grupos de acesso e cadastro e edição de automações, conforme ilustrado pela Figura 3. Além disso, também podem executar automações para fins de teste e *debug*. A ideia é que esse tipo de usuário seja responsável pelo cadastro de outros usuários e também pelo cadastro e configuração de novas automações. Ainda na Figura 3, mencionada anteriormente, os botões “Adicionar” e “Modificar” ao lado dos itens das tabelas indicam que o perfil possui permissões de escrita.

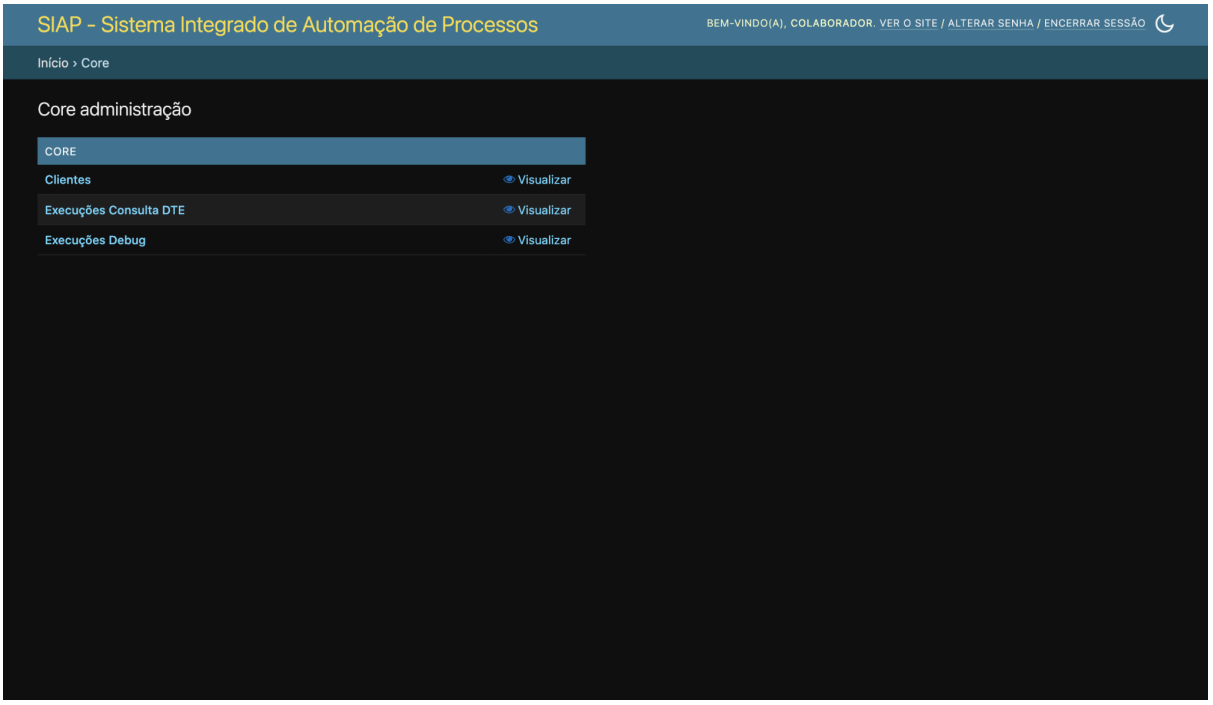
Figura 4 - Tela inicial para o usuário Gestor



Fonte - Elaborado pelo autor (2025)

Usuários do tipo Gestor possuem um acesso mais restrito, isto é, com menos poderes, do que os do tipo Desenvolvimento/Suporte. A Figura 4 ilustra as funcionalidades disponíveis para esse tipo de usuário, que são aquelas relacionadas à gestão dos clientes, incluindo as ações de cadastro e edição, caracterizadas pela presença dos botões “Adicionar” e “Modificar” na linha referente ao modelo Clientes, e as de acessar e executar as automações, como pode ser percebido pela presença dos modelos de execução, como o Executar Consulta DTE.

Figura 5 - Tela inicial para o usuário Colaborador



Fonte - Elaborado pelo autor (2025)

Por fim, a Figura 5 exibe a tela dos usuários do tipo Colaborador, aqueles que devem ser os mais numerosos dentro do sistema. Nesse caso, possuem ainda menos privilégios que os usuários do tipo Gestor, podendo somente visualizar os clientes cadastrados – possível perceber pela ausência dos botões de ação “Adicionar” e “Modificar” – e executar as automações. A intenção é que esse usuário de fato opere a rotina de execução de automações e usufrua dos seus resultados, já que, para a grande maioria dos processos automatizados, esse perfil de usuário, o colaborador, é quem realizava, de forma manual, a rotina descrita pelo processo.

Seguindo com o fluxo de execução manual e partindo da tela exibida na captura da Figura 5, ao clicar em uma das opções de Execuções de automações, o usuário navegará para uma página na qual poderá selecionar os clientes cadastrados na referida automação e enviar uma solicitação de execução contemplando esses clientes selecionados. Essa tela será ilustrada a seguir, na Figura 6, demonstrando o cenário em que o usuário clicou em “Execuções Consulta DTE”.

Figura 6 - Tela de execução manual da automação Consulta DTE

Fonte - Elaborado pelo autor (2025)

A tela de seleção de clientes, exibida na Figura 6 logo acima, apresenta uma tabela com a listagem dos clientes cadastrados para os quais a automação do processo Consulta DTE está disponível. Além disso, destacado em amarelo acima da tabela, existe um menu *dropdown* em que o usuário deve selecionar a opção "Executar Automação" e, então, clicar no botão "Ir", solicitando que a execução seja publicada no sistema de mensageria (*Pub/Sub*).

Ainda sobre a tabela da Figura 6, a primeira coluna (CLIENTE) e as duas últimas (ESTADO ATUAL e ÚLTIMO ESTADO) terão seus valores padronizados para todas as automações, sendo a primeira coluna a razão social do cliente e as duas últimas o estado da solicitação de execução atual, caso exista alguma, e o estado da última solicitação de execução, respectivamente. Os valores que podem ser assumidos pelas colunas de estado são "Pendente", "Em Execução", "Concluída" e "Erro". As demais colunas, que contemplam a parte central da tabela – MENSAGENS NÃO LIDAS e MENSAGENS LIDAS, neste caso – serão específicas de cada automação. Para esta automação específica, tais colunas armazenam os dados extraídos na execução do script de automação e recebidos via API REST pelo *endpoint* de atualização, permitindo que o usuário saiba quais empresas possuem mensagens não lidas no Domicílio Tributário Eletrônico (DTE), consultado no portal da Unidade Virtual de Tributação do Estado do Rio Grande do Norte (RN).

Ao executar essa ação, como mencionado há pouco, o sistema publicará uma mensagem no *Pub/Sub* que será consumida pelos *Bot Runners*. O trecho de código abaixo mostra a função que é invocada para realizar o processo de publicação da mensagem com os dados da solicitação de execução no tópico do sistema de mensageria:

```
@admin.action(description="Executar Automação")
def execute_automation(
    modeladmin: admin.ModelAdmin,
    request: HttpRequest,
    executions: QuerySet[BaseExecution],
):
    execution = executions.first()
   if execution is None:
        return

    payload = {
        "automation": AutomationSerializer(execution.automation).data,
        "executions": ExecutionRequestSerializer(executions, many=True).data,
    }

    attrs = {
        "process_name": execution.automation.process,
    }

    logging.info(f"Payload: {json.dumps(payload)}")

    try:
        pubsub = PubSub(
            project_id=settings.GCP_PROJECT_ID,
            topic=execution.automation.topic_name,
        )
        pubsub.publish(payload=payload, attrs=attrs)
        execution.set_all_as_pending(executions)

    except Exception as e:
        logger.error(
            "An unexpected error occurred when publishing execution requests for"
            f" {str(execution.automation)} automation. Error: {str(e)}",
        )
        execution.set_all_as_error(executions)
```

Nos parâmetros da função `execute_automation` definida acima, o `executions` é exatamente a coleção que armazena os dados dos clientes selecionados na tabela da tela de execução, exibida na Figura 6, e o `payload` (carga útil) da publicação é composto pelos dados da automação junto com os dados das `executions`, que armazenam as informações dos clientes, ambos serializados – convertidos para uma representação de cadeia de caracteres de fácil transmissão. Outro ponto que vale a pena destacar é o bloco `try/except` para tratar possíveis exceções, como erros de rede, por exemplo, durante o processo de publicação. Dessa forma, se qualquer erro inesperado ocorrer durante a instanciação do cliente `PubSub` ou durante a publicação da mensagem, o estado dos clientes selecionados na tabela mudará

imediatamente para “Erro” e o usuário saberá que sua solicitação não pode ser concluída com êxito.

Ao finalizar a execução de cada um dos clientes, o *Bot Runner* envia atualizações de status via requisição HTTP, do tipo POST, para um *endpoint* REST implementado na aplicação *Django*, permitindo que os usuários visualizem o estado atual de cada solicitação de execução realizada. A implementação desse *endpoint* envolve o desenvolvimento de alguns componentes, mas, dentre eles, os principais são o *serializer*, responsável por converter os dados recebidos em formato JSON para objetos Python nativos, e uma *viewset*, que é uma classe responsável por receber a requisição, coordenar o seu processamento e devolver uma resposta. Os trechos de código a seguir mostram a implementação do *serializer*:

```
class ExecutionUpdateSerializer(serializers.Serializer):
   current_state = serializers.ChoiceField(choices=BaseExecution.StateChoices)

   def update(self, instance: BaseExecution, validated_data: dict):
       self._update_current_state(instance, validated_data)
       return instance

   def _update_current_state(self, instance: BaseExecution, validated_data: dict):
       update_status_method_map = {
           BaseExecution.StateChoices.PENDING_STATE: instance.set_as_pending,
           BaseExecution.StateChoices.IN_EXECUTION_STATE:
instance.set_as_in_execution,
           BaseExecution.StateChoices.DONE_STATE: instance.set_as_done,
           BaseExecution.StateChoices.ERROR_STATE: instance.set_as_error,
       }

       new_state = validated_data.get("current_state")
       update_method = (
           update_status_method_map.get(
               BaseExecution.StateChoices(new_state), instance.set_as_empty
           )
           if new_state is not None
           else instance.set_as_empty
       )
       if update_method:
           update_method(commit=True)

class ConsultaDTEExecutionUpdateSerializer(ExecutionUpdateSerializer):
   unread_messages = serializers.IntegerField()
   read_messages = serializers.IntegerField()

   def update(self, instance: ConsultaDTEExecution, validated_data: dict):
       super().update(instance, validated_data)
       instance.unread_messages = validated_data.get(
           "unread_messages", instance.unread_messages,
       )
       instance.read_messages = validated_data.get(
           "read_messages", instance.read_messages,
       )
       instance.save()

       return instance
```

Cada automação precisará de um serializer específico que herda do serializer base, pois elas terão campos diferentes que receberão valores diferentes.

Dando continuidade, o trecho de código abaixo mostra a implementação da *viewset* responsável por receber, processar e responder às requisições de atualização de estado recebidas a partir dos *Bot Runner*:

```
class BaseExecutionViewSet(mixins.UpdateModelMixin, viewsets.GenericViewSet):
   lookup_field = "uuid"

   def partial_update(self, request, *args, **kwargs):
       instance = self.get_object()
       serializer = self.get_serializer(instance, data=request.data, partial=True)
       serializer.is_valid(raise_exception=True)
       self.perform_update(serializer=serializer)

       return Response(status=HTTP_204_NO_CONTENT)

class ConsultaDTEExecutionViewSet(BaseExecutionViewSet):
   serializer_class = ConsultaDTEExecutionUpdateSerializer
   queryset = ConsultaDTEExecution.objects.all()
```

Assim como para os *serializers*, cada automação também precisa de uma *viewset* específica, uma vez que elas utilizam os *serializers* para preencher o valor do atributo `serializer_class`. Tendo em vista essa necessidade de repetir a criação desses componentes para cada uma das automações e pensando num cenário futuro otimista em que vários outros processos são automatizados a partir deste sistema, utilizou-se bastante no projeto práticas da Orientação a Objeto como a herança, através da criação classes base, como `ExecutionUpdateSerializer` e `BaseExecutionViewSet`, para abrigar os comportamentos e atributos comuns à todas as implementações desses componentes e evitar duplicação desnecessária de código. Essa abordagem foi amplamente utilizada ao longo do desenvolvimento do projeto, não somente na construção do *endpoint* REST, mas também na implementação dos modelos e todas as outras estruturas para as quais fosse pertinente.

3.2.2 Bot Runner FastAPI

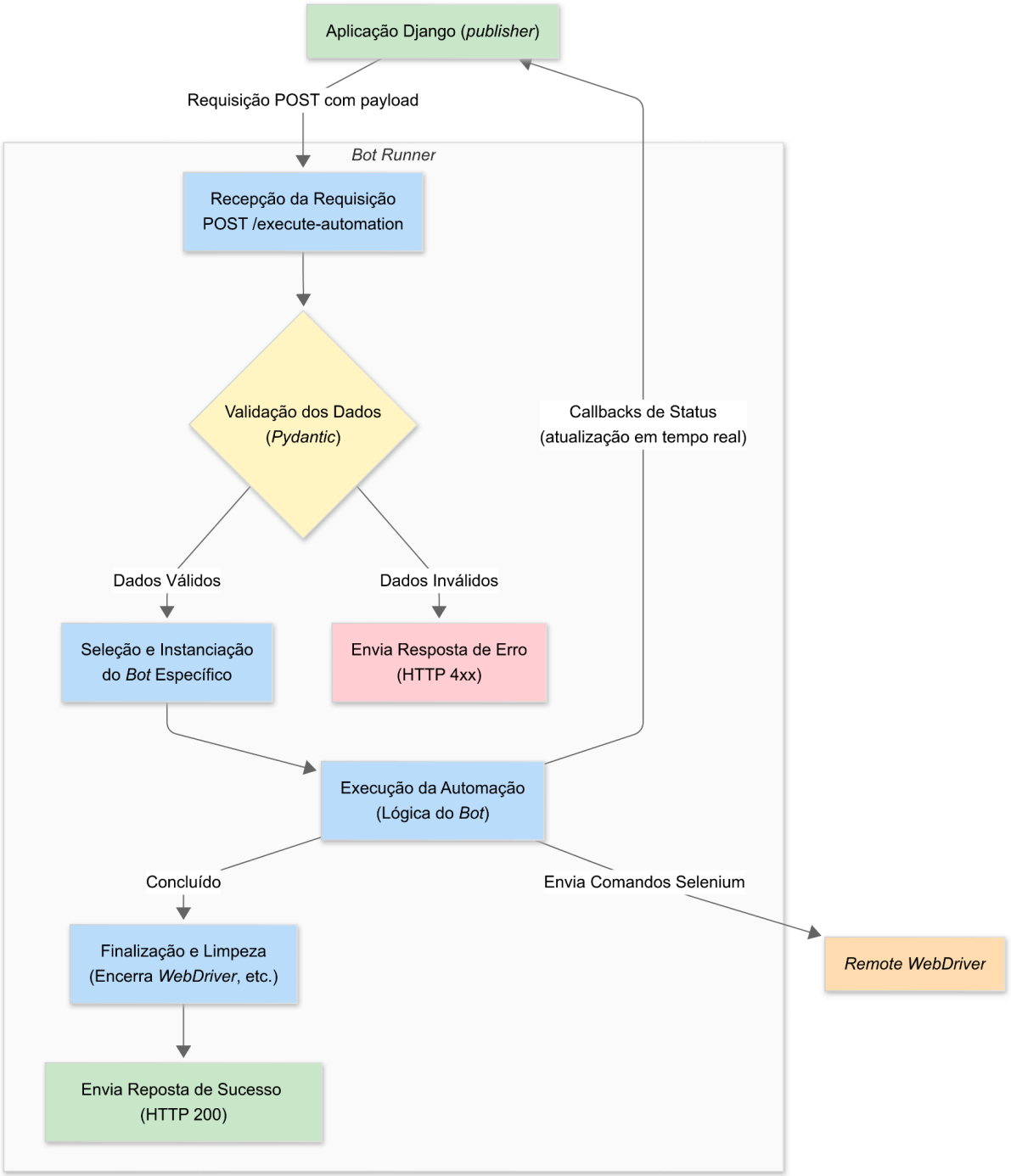
Componente cuja responsabilidade é receber e processar as solicitações de execução de automação, utilizando o framework FastAPI para expor endpoints HTTP e o Selenium para interagir de forma automatizada com sistemas web. Em conjunto com a aplicação *Django*, apresentada nos tópicos anteriores, constitui uma das peças centrais na arquitetura desenvolvida para a solução. Este serviço opera de forma desacoplada como um *subscriber* (ou assinante, em tradução literal), consumindo mensagens do *Pub/Sub* e reportando o status das execuções para a aplicação que as publicou, permitindo que os serviços escalem de forma independente e eficiente.

Assim como a aplicação *Django*, o *Bot Runner* é implementado como um serviço empacotado em uma imagem *Docker* personalizada, executada no *Google Cloud Run*. Ele se comunica com outros componentes do sistema por meio de *APIs REST* e *Pub/Sub*, além de interagir com um serviço de *WebDriver* remoto do *Selenium*, também hospedado em ambiente cloud, que executa as ações de forma automatizada no navegador web.

A estrutura do projeto segue uma organização modular, separando as responsabilidades em camadas como DTOs (*Data Transfer Objects*), *scripts* de automação, utilitários, e roteadores de *API*.

3.2.2.1 Fluxo de Execução

Figura 7 - Fluxo de execução da aplicação *Bot Runner*



Fonte - Elaborado pelo autor (2025)

A Figura 7 ilustra o fluxo de execução do *Bot Runner*, que pode ser resumido nas etapas descritas a seguir.

1. Recebimento da Requisição de Execução: o *Bot Runner* disponibiliza um endpoint HTTP específico (`/execute-automation`), que recebe requisições do tipo POST originadas, tipicamente, pela aplicação principal (*Django*). Essas requisições contêm um payload estruturado, incluindo informações detalhadas sobre os clientes, o processo da automação a ser executada e, quando necessário, credenciais de acesso, conforme ilustrado a seguir:

```
{
    "automation": {
        "uuid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
        "process": "consulta-dte",
        "application": {
            "name": "UVT",
            "username": "login-credencial-uvt",
            "password": "senha-credencial-uvt"
        }
    },
    "executions": [
        {
            "uuid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
            "client": {
                "uuid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
                "razao_social": "Cliente PJ 1",
                "email": "clientepj1@email.com",
                "cnpj": "00000000000001",
                "cpf": null
            }
        }
    ]
}
```

Abaixo, segue trecho de código que implementa a lógica do rota descrita, que é a porta de entrada da aplicação, responsável por orquestrar todo restante do processo:

```
@router.post("/execute-automation")
async def execute_automation(execution_input: ExecutionInput):
    bot = bot_factory.get_bot(execution_input=execution_input)
    if not bot:
        process = execution_input.automation.process.value
        return JSONResponse(
            content={"detail": f"bot not implemented for process '{process}'"},
            status_code=status.HTTP_400_BAD_REQUEST,
        )

    try:
        await asyncio.to_thread(bot.execute)
        return Response(status_code=status.HTTP_204_NO_CONTENT)

    except Exception as exc:
        logger.warning(
            "an exception ocurred when executing"
            f" {execution_input.automation.process} automation: {exc}",
        )
        return await handle_execution_exception(input=execution_input, bot=bot)
```

2. Validação e Preparação: ao receber a requisição, o serviço realiza uma validação rigorosa dos dados utilizando modelos definidos com a biblioteca *Pydantic* – ferramenta muito difundida entre a comunidade *Python* para lidar com validação de dados. Essa validação abrange desde a verificação de formatos e tipos (como CNPJ, CPF, datas e e-mails) até a checagem de obrigatoriedade e consistência dos campos. Os modelos responsáveis por essa validação são os seguintes:

```
class Process(str, Enum):
    consulta_dte = "consulta-dte"


class Application(BaseModel):
    name: str
    username: str | None
    password: str | None


class Automation(BaseModel):
    uuid: str
    process: Process
    application: Application


class Client(BaseModel):
    uuid: str
    company_name: str
    email: str
    cnpj: str | None
    cpf: str | None

    @model_validator(mode="after")
    def validate_cnpj_or_cpf(self):
        cnpj_filled = self.cnpj is not None
        cpf_filled = self.cpf is not None

        if cnpj_filled and cpf_filled:
            raise ValueError("clients should only have CNPJ or CPF, never both.")

        if not cnpj_filled and not cpf_filled:
            raise ValueError("clients should have at least one of CNPJ or CPF.")

        if self.cnpj and (not self.cnpj.isdigit() or len(self.cnpj) != 14):
            raise ValueError("CNPJ should have exactly 14 numeric digits.")

        if self.cpf and (not self.cpf.isdigit() or len(self.cpf) != 11):
            raise ValueError("CPF should have exactly 11 numeric digits.")

        return self


class Execution(BaseModel):
    uuid: str
    client: Client


class ExecutionInput(BaseModel):
    automation: Automation
    executions: list[Execution]
```

Caso algum dado esteja em desacordo com as regras estabelecidas, a requisição é rejeitada com uma resposta detalhada sobre os erros encontrados.

3. Seleção e Instanciação do Bot: com os dados validados, o *Bot Runner* identifica qual *script* de automação deve ser utilizado, de acordo com o processo solicitado e instancia a classe que implementa a automação do respectivo processo. Essa seleção e instanciação é implementada pelo seguinte trecho de código:

```
class BotFactory:
    def __init__(
        self,
        bot_class_map: BotClassMap,
        bot_kwargs_factory_map: BotKwargsFactoryMap,
    ) -> None:
        self._bot_class_map = bot_class_map
        self._bot_kwargs_factory_map = bot_kwargs_factory_map

    def get_bot(self, execution_input: ExecutionInput) -> BaseBot | None:
        process = execution_input.automation.process

        bot_class = self._bot_class_map.get(process)
        if not bot_class:
            return None

        update_callback = get_update_callback(process)

        bot_kwargs = {
            "executions": execution_input.executions,
            "update_callback": update_callback,
        }
        bot_kwargs_factory = self._bot_kwargs_factory_map.get(process)
        if bot_kwargs_factory:
            bot_kwargs.update(bot_kwargs_factory(execution_input))

        return bot_class(**bot_kwargs)
```

A classe descrita acima, trata-se de uma *factory* (ou fábrica, em tradução livre) de *bots*. Esse padrão de projeto é muito utilizado para implementar componentes responsáveis por abstrair a criação de objetos complexos. Uma vez identificada a classe do *bot* apropriada (por exemplo, `ConsultaDteBot`), o serviço instancia a função de *callback*, que será utilizada para atualização dos clientes durante a execução, e busca também pelos parâmetros de inicialização do *bot*, ainda com base no processo fornecido. Por fim, instancia a classe do *bot* correspondente, injetando os parâmetros necessários, como dados do cliente, credenciais e eventuais configurações específicas do processo. Esse mecanismo garante flexibilidade e extensibilidade, permitindo a inclusão de novos *bots* de forma simples e organizada.

4. Execução da Automação: o bot selecionado utiliza o *Selenium* para interagir com o navegador remoto, realizando todas as ações necessárias para cumprir o objetivo da automação, como login em sistemas, navegação por páginas, preenchimento de formulários e extração de dados. Essas interações ocorrem por meio de requisições HTTP para uma aplicação executando um servidor *WebDriver Selenium* de forma remota em uma instância própria do *Cloud Run*. Dado que o fluxo de execução das automações será sempre muito semelhante, a seguinte classe base, responsável por implementar esse processamento, foi desenvolvida:

```
class BaseBot(ABC):
    def __init__(
        self,
        executions: list[Execution],
        update_callback: Callable[
            [str, BaseOutput], CoroutineType[Any, Any, None]
        ],
    ) -> None:
        self._executions = executions
        self._update_callback = update_callback
        self._current_execution_index: int

        options = Options()
        self._driver = webdriver.Remote(
            command_executor=settings.SELENIUM_COMMAND_EXECUTOR,
            options=options,
        )
        self._wait = WebDriverWait(
            self._driver,
            settings.SELENIUM_WAIT_TIMEOUT_SECONDS,
        )

    async def execute(self):
        try:
            await self.run_process()
        except Exception:
            if self._current_execution_index:
                for execution in self._executions[self._current_execution_index:]:
                    await self.error(execution.uuid)
        finally:
            try:
                self._teardown()
            except Exception as e:
                pass

    @abstractmethod
    def run_process(self):
        raise NotImplementedError("should be implemented in bot classes")

    def _teardown(self):
        self._driver.quit()
```

A classe `BaseBot` constitui a fundação para todos os robôs de automação do sistema. Implementada como uma Classe Base Abstrata (ABC), ela não é utilizada diretamente, mas serve como um "molde" que define a estrutura e o comportamento padrão que todas as

classes de bots concretas devem herdar. O método `execute` define o algoritmo geral de uma execução, enquanto delega os passos específicos para as subclasses através do método abstrato `run_process`.

No momento de sua instanciação, a classe recebe os parâmetros essenciais para a execução, como a lista de tarefas e uma função de *callback* assíncrona (`update_callback`), utilizada para comunicar atualizações de estado em tempo real para a aplicação principal. O construtor é também responsável por inicializar a conexão com *WebDriver* remoto do *Selenium*, apontando para o executor de comandos configurado no sistema, e por instanciar um *WebDriverWait*, garantindo uma interação mais robusta e resiliente com os elementos dinâmicos das páginas web.

O método principal, `execute`, orquestra todo o ciclo de vida da automação. Toda a execução é encapsulada em um bloco `try/finally` para garantir que a rotina de finalização e limpeza de recursos, implementada pelo método `_teardown`, seja sempre executada, mesmo em caso de falhas. No caso de exceções serem lançadas, dentro do método `run_process`, elas serão levantadas e até o nível da função `execute_automation` que implementa a rota e serão devidamente tratadas pela função `handle_execution_exception`, responsável por implementar uma lógica customizada de *retry* (retentativa):

```
async def handle_execution_exception(input: ExecutionInput, bot: BaseBot):
    remaining_executions = input.executions[bot.current_execution_index :]

    if input.retries_left == 0:
        coroutines = [bot.error(exec.uuid) for exec in remaining_executions]
        await asyncio.gather(*coroutines)

        return JSONResponse(
       content={
                "detail": "retry limit exceeded. ending execution with error"
            },
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        )

    if remaining_executions:
        logger.warning("Pending executions were found. Retrying...")

        retry_input = ExecutionInput(
       automation=input.automation,
            executions=remaining_executions,
            retries_left=input.retries_left - 1,
        )
        pubsub = PubSub(
            settings.GCP_PROJECT_ID, settings.GCP_PUBSUB_DEFAULT_TOPIC
        )
        await pubsub.publish(retry_input.model_dump(), {"retry": "true"})

    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

A função verifica se o limite máximo de tentativas ainda não foi atingido e se existem clientes cuja execução ainda está pendente dentro da solicitação. Se ambas as condições forem verdadeiras, publica uma nova mensagem no *Pub/Sub* solicitando que uma nova tentativa de execução somente com aqueles clientes que ficaram pendentes, evitando que os clientes para os quais a execução foi realizada corretamente sejam reprocessados sem necessidade.

5. Finalização e Limpeza: após a conclusão da automação para todos os clientes, seja com sucesso ou em caso de erro, o bot executa rotinas de finalização, que incluem o encerramento do *WebDriver* e a liberação de recursos alocados durante o processo.

3.2.3 Implementação da Automação Piloto: Consulta ao DTE na UVT

O primeiro processo de negócio selecionado para automação e validação da plataforma foi a consulta ao Domicílio Tributário Eletrônico (DTE) no portal da Unidade Virtual de Tributação (UVT) do estado do Rio Grande do Norte (RN). O DTE funciona como ambiente digital para o qual são encaminhadas ao contribuinte as comunicações expedidas pela Receita Estadual. Dentre essas comunicações, encontram-se as Notificações ou Intimações, os Avisos, os comunicados para Autorregularização e os Recibos relacionados ao cumprimento das obrigações acessórias (RECEITA-RS).

A escolha deste processo como caso de uso inicial foi estratégica e baseada em múltiplos fatores. Primeiramente, trata-se de um processo de curta duração – aproximadamente um minuto para consultar o primeiro cliente, devido a necessidade de realização do login, e mais meio minuto para cada cliente adicional – e com um fluxo de implementação simples, envolvendo poucas etapas e um portal de fácil interação, o que mitiga os riscos de falhas inesperadas e facilita o desenvolvimento. Adicionalmente, ele apresenta um alto potencial de retorno sobre o investimento (ROI), uma vez que a consulta deve ser realizada diariamente para toda a base de clientes do escritório, e a automação de uma tarefa que, manualmente, consome um tempo acumulado significativo, gera uma grande economia de horas de trabalho que podem ser realocadas para atividades mais analíticas. Somado a tudo isso, junta-se o fato de que manter essa consulta em dia é de extrema importância para garantir a conformidade fiscal das empresas. A seguir, será apresentado o fluxo de execução manual da Consulta ao DTE e todo seu processo de automatização.

3.2.3.1 Fluxo do Processo

A execução manual do processo de consulta ao DTE, realizada por um colaborador, segue uma sequência de passos repetitivos, conforme detalhado a seguir:

1. Acessar o portal da UVT e fazer login com as credenciais (usuário e senha);
2. Buscar na base de dados do escritório (geralmente uma planilha) o CNPJ ou CPF da empresa cujo DTE será consultado;
3. Selecionar a empresa no portal da UVT utilizando o CNPJ ou CPF;
4. Clicar no botão "Consultar DTE" na tela inicial da aplicação;
5. Aguardar a abertura do modal com as informações das mensagens de DTE e verificar se existem mensagens não lidas;
6. Fechar o modal da consulta e repetir os passos de 2 a 5 para todos os clientes da planilha.

3.2.3.2 Implementação do Bot Consulta DTE

O *bot* a seguir automatiza o fluxo manual descrito anteriormente:

```
class ConsultaDteBot(BaseBot):
    APP_URL = "https://uvt.set.rn.gov.br/"

    def __init__(
        self,
        executions: list[Execution],
        update_callback: Callable[
            [str, BaseOutput], CoroutineType[Any, Any, None],
        ],
        username: str,
        password: str,
    ) -> None:
        super().__init__(executions=executions, update_callback=update_callback)
        self._username = username
        self._password = password

    async def run_process(self):
        self.access_uvt()
        self.login_with_credentials()

        for index, _ in enumerate(self._executions):
            self.set_current_execution(index=index)
            await self.start(execution_uuid=self.current_execution.uuid)
            self.select_company(
                self.current_execution.client.cnpj
                or self.current_execution.client.cpf
            )
            result = self.consult_dte(self.current_execution.client.uuid)
            result.set_as_done()

            await self.success(self.current_execution.uuid, result)
```

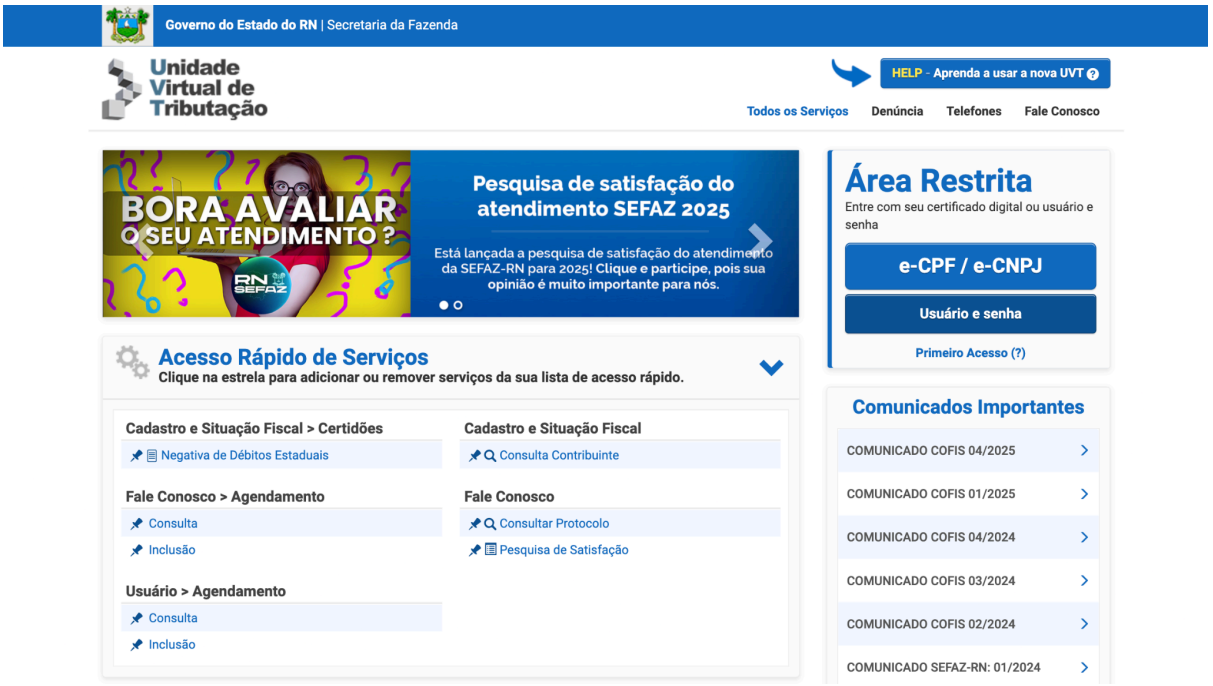
A classe `ConsultaDteBot` herda a estrutura e o comportamento definidos em `BaseBot`, implementando a lógica específica para a consulta ao DTE. O método `run_process` desta classe orquestra uma sequência de ações, delegando cada etapa a métodos auxiliares que encapsulam uma responsabilidade bem definida. A seguir, serão descritos em mais detalhe os quatro principais métodos que compõem este fluxo, os quais foram destacados em negrito no código acima.

Iniciando pelo `access_uvt`, este método é o ponto de partida da interação, sendo responsável por uma única e crucial tarefa: direcionar o navegador para a URL principal do portal da UVT:

```
def access_uvt(self):
        self._driver.get(self.APP_URL)
        self._wait.until(ec.element_to_be_clickable(self.BTN_USUARIO_SENHA))
```

Além de simplesmente navegar para o endereço, ele utiliza um *WebDriverWait* para aguardar de forma explícita até que o botão de login esteja disponível para clique. Essa espera garante que a página tenha sido completamente carregada e que seus elementos interativos estejam prontos, tornando a execução do robô mais resiliente a variações no tempo de carregamento da página. A figura a seguir mostra a página inicial do portal da UVT:

Figura 8 - Tela inicial do portal da UVT antes do *login*



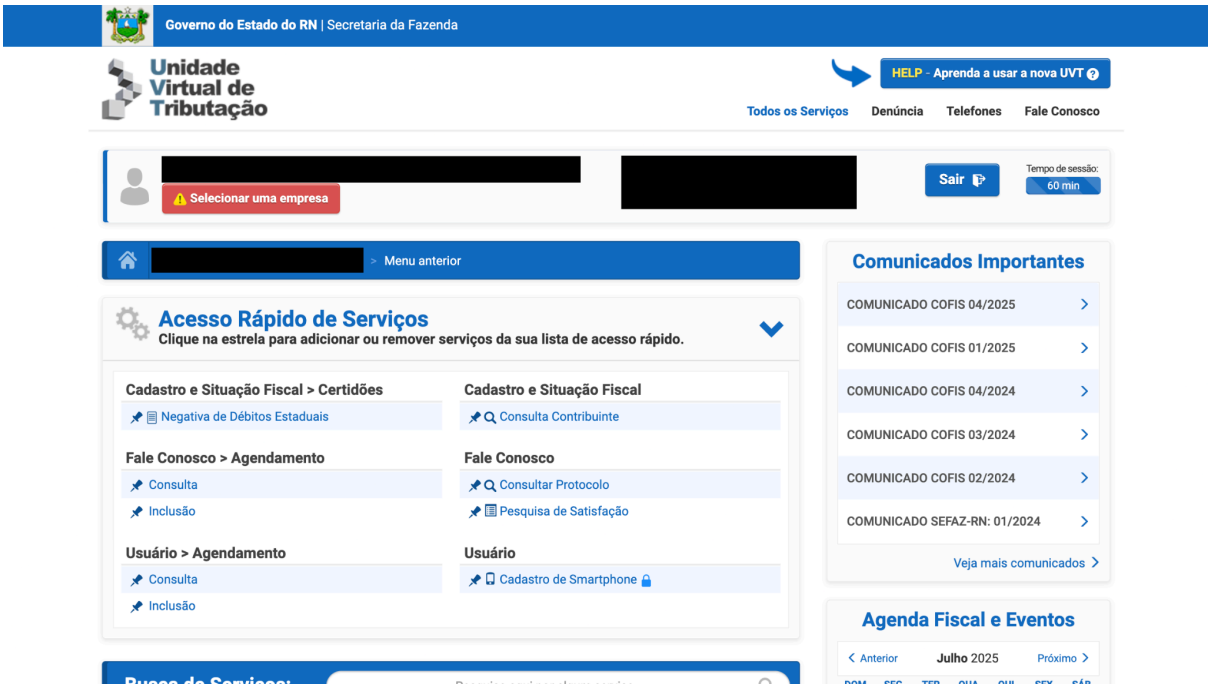
Fonte - Elaborado pelo autor (2025)

Com a página carregada, conforme ilustrado pela Figura 8, inicia-se a etapa de autenticação, na qual o *bot* seleciona o meio pelo qual realizará o processo – via Usuário e Senha, neste caso – e preenche os campos de código de acesso e senha, conforme descrito a seguir:

```
def login_with_credentials(self):
    self._wait.until(ec.element_to_be_clickable(self.BTN_USUARIO_SENHA)).click()

    self._wait.until(
        ec.element_to_be_clickable(self.INPUT_CODIGO)
    ).send_keys(self._username)
    self._wait.until(
        ec.element_to_be_clickable(self.INPUT_SENHA)
    ).send_keys(self._password)

    self._wait.until(ec.element_to_be_clickable(self.BTN_ACESSAR)).click()

    self._wait.until(ec.element_to_be_clickable(self.BTN_SELECIONAR_EMPRESA))
```

Ao final, o método aguarda o aparecimento de um elemento, o botão “Selecionar uma empresa”, da página seguinte para confirmar que o acesso foi bem-sucedido.

Figura 9 - Tela inicial do portal da UVT após o *login*



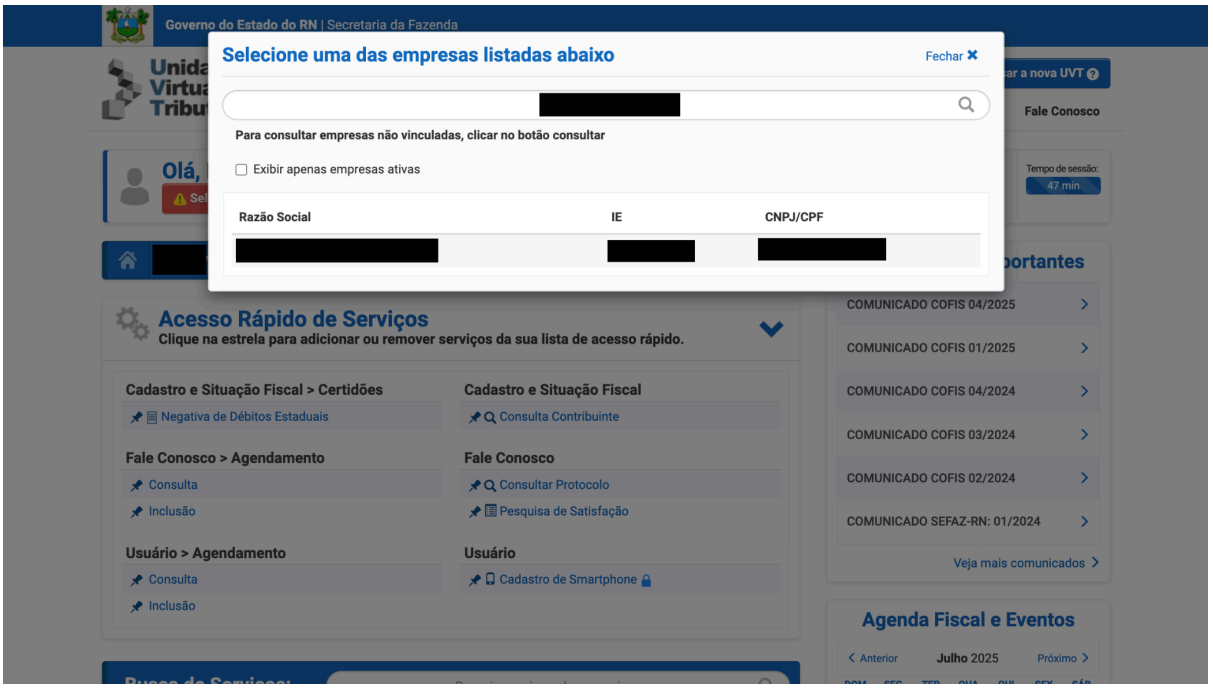
Fonte - Elaborado pelo autor (2025)

Uma vez autenticado na plataforma, a tela da Figura 9 será exibida. O próximo método a ser descrito, o `select_company`, será invocado para alternar o contexto da sessão para o cliente desejado:

```
def select_company(self, cnpj_ou_cpf):
    self._wait.until(
        ec.element_to_be_clickable(self.BTN_SELECIONAR_EMPRESA)
    ).click()
    self._wait.until(
        ec.element_to_be_clickable(self.INPUT_PESQUISAR_EMPRESA)
    ).send_keys(cnpj_ou_cpf)
    self._wait.until(
        ec.element_to_be_clickable(self.LINK_EMPRESA_ENCONTRADA)
    ).click()
    self._wait.until(ec.element_to_be_clickable(self.BTN_DTE))
```

Ele recebe o CNPJ ou CPF como parâmetro, aciona o botão “Selecionar uma empresa” e busca e busca pelo valor recebido no campo de texto no modal para encontrar a empresa. Então, clica no resultado correspondente. Logo abaixo, a Figura 10 ilustra o modal de seleção de empresa:

Figura 10 - Modal de seleção de empresas



Fonte - Elaborado pelo autor (2025)

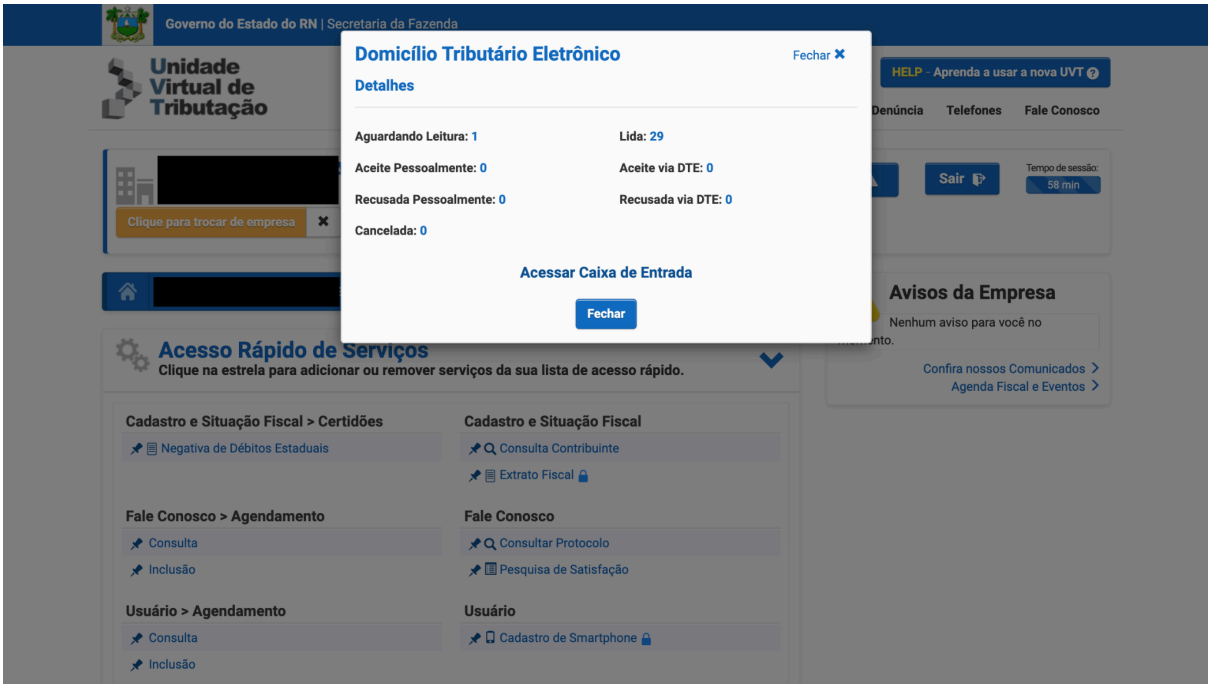
Assim como nas etapas anteriores, o método finaliza com uma espera explícita por um elemento da tela principal do cliente, o botão “DTE”, assegurando que a seleção foi concluída com sucesso antes de permitir que o fluxo prossiga para a próxima ação.

Por fim, o método `consult_dte` executa a ação principal e o objetivo final do robô que é, como o nome do método sugere, acessar as informações do DTE para verificar se existem mensagens não lidas para a respectiva empresa selecionada:

```
def consult_dte(self, id_cliente) -> ConsultaDTEOutput:
    self._wait.until(ec.element_to_be_clickable(self.BTN_DTE)).click()
    self._wait.until(ec.element_to_be_clickable(self.BTN_FECHAR))
    modal_dte = self._wait.until(ec.element_to_be_clickable(self.MODAL_DTE))
    modal_dte_h5_elements = modal_dte.find_elements(By.TAG_NAME, "h5")
    modal_dte_text = [e.text for e in modal_dte_h5_elements]

    raw_data = {}
    for data in modal_dte_text:
        key, value = data.split(":")
        key, value = key.strip(), value.strip()
        raw_data[key] = value

    self._wait.until(ec.element_to_be_clickable(self.BTN_FECHAR)).click()

    self._wait.until(ec.element_to_be_clickable(self.BTN_SELECIONAR_EMPRESA))

    return ConsultaDTEOutput(
        unread_messages=int(raw_data["Aguardando Leitura"]),
        read_messages=int(raw_data["Lida"]),
    )
```

Ele aciona o botão de consulta ao DTE, aguarda a exibição do modal de informações, e então realiza a extração dos dados. A seguir, a Figura 11 contempla a imagem do modal no qual estão as informações acerca das do DTE.

Figura 11 - Modal de status do DTE



Fonte - Elaborado pelo autor (2025)

A lógica de parsing percorre os elementos de texto dentro do modal, separa os indicadores (como "Aguardando Leitura" e "Lida") de seus respectivos valores numéricos e, após fechar a janela, retorna essas informações em um objeto de dados estruturado, o `ConsultaDTEOutput`, que será posteriormente enviado de volta à aplicação principal via requisição HTTP com os dados extraídos, implementada na *callback* de atualização.

Vale salientar que não compete ao *bot* a leitura da mensagem, e sim somente a consulta para verificar sua existência. A leitura das mensagens e ações que devem ser tomadas em decorrência ao conteúdo continuam sendo de inteira responsabilidade do colaborador.

Dessa forma, o *bot* finaliza todas as etapas, antes realizadas de forma manual, e retorna os resultados para a plataforma *Django* na qual os usuários poderão visualizar, de modo muito mais eficiente, os resultados de todos os clientes em uma única tabela.

4 Resultados Obtidos

Este capítulo apresenta os resultados obtidos durante o período de validação prática da plataforma de automação. O foco desta etapa foi avaliar a estabilidade operacional da solução em um ambiente real, identificar desafios técnicos na execução das automações e quantificar os ganhos de eficiência gerados. Os dados coletados e os eventos observados ao longo de três semanas de testes são detalhados e analisados a seguir.

A metodologia de validação foi conduzida em um cenário de uso prático no escritório parceiro, ao longo de três semanas no mês de junho de 2025. Adotou-se um modelo de implantação gradativa, no qual a confiança nos resultados automatizados foi construída progressivamente, com um colaborador responsável validando manualmente todas as consultas realizadas pelo robô. Diariamente, o processo era disparado para a base de 92 clientes, e o comportamento do sistema era monitorado. Essa abordagem permitiu não apenas a verificação da corretude dos dados, mas também a observação do desempenho da arquitetura sob carga e a aplicação de ajustes técnicos de forma iterativa, com base nos logs e eventos registrados.

Os principais eventos e resultados observados durante o período de validação foram compilados e organizados no conjunto de tabelas abaixo, que detalha a evolução da estabilidade do sistema, os desafios encontrados e soluções encontradas.

SEMANA (DATA)	**OBSERVAÇÕES E EVENTOS RELEVANTES**
SEMANA I (02/06)	● 25% da base de clientes (23 empresas); ● Instabilidade inicial. O mecanismo de *retry* funcionou, realizando as 5 tentativas de execução conforme programado, mas não foi suficiente e foram necessárias 3 solicitações de execução para processar os 23 clientes. Ou seja, o *retry* foi excedido por duas vezes. A causa foi atribuída à insuficiência de recursos no *Remote WebDriver*; ● **15** mensagens não lidas encontradas.
SEMANA I (03/06)	● Os recursos do Cloud Run (RAM/vCPU) foram aumentados.

	A instabilidade diminuiu, mas o *retry* ainda não foi suficiente e foram necessárias 2 solicitações para consultar os 23 clientes. Identificou-se a necessidade de configurações específicas do *Selenium* para ambientes de contêiner (`--disable-dev-shm-usage`, por exemplo); ● **13** mensagens não lidas encontradas.
SEMANA I (04/06 - 06/06)	● Após o ajuste de recursos e a aplicação das novas configurações no *WebDriver*, a plataforma atingiu estabilidade e foi capaz de consultar os 23 clientes em uma única solicitação; ● A quantidade de mensagens não lidas diminuiu gradativamente até chegar em **3**.
SEMANA II (09/06 - 11/06)	● Aumento para 50% da base de clientes (46 empresas); ● O comportamento estável se manteve, confirmando a eficácia das correções implementadas. Todos os clientes foram consultados em uma única solicitação; ● A quantidade de mensagens não lidas continuou diminuindo gradativamente até zerar, no dia 11.
SEMANA II (12/06)	● Nenhuma execução foi realizada. Ocorreu uma instabilidade global na *Google Cloud Platform* que tornou os serviços indisponíveis no momento da execução.
SEMANA II (13/06)	● Os serviços da GCP foram normalizados e a execução voltou a ocorrer sem problemas, inclusive mantendo o comportamento de estabilidade presenciado nos dias anteriores, consultando todas as empresas em uma única solicitação; ● Não foram encontradas mensagens não lidas.
SEMANA III (16/06 - 18/06)	● Aumento para 100% da base de clientes (92 empresas); ● A estabilidade operacional da plataforma se manteve

	consistente. Mesmo com uma quantidade de clientes muito maior que as anteriores, o processo de consulta foi executado completamente com uma única solicitação; ● Não foram encontradas mensagens não lidas.
SEMANA III (19/06)	● Feriado nacional de Corpus Christi.
SEMANA III (20/06)	● A execução final do período de testes ocorreu de forma estável e seguiu concluindo todas as consultas com uma única solicitação; ● **2** novas mensagens não lidas foram encontradas.

Quadro 3 - Resultados observados

A análise dos resultados demonstra uma clara curva de maturação da plataforma. A fase inicial de testes foi crucial para expôr e solucionar desafios técnicos relacionados tanto à alocação de recursos de infraestrutura quanto à configuração fina do software em um ambiente de contêineres. Após os ajustes realizados na primeira semana, o sistema provou ser robusto e confiável, operando de forma estável e previsível.

Além da validação técnica, a automação demonstrou seu valor prático ao identificar consistentemente mensagens importantes para os clientes. Considerando os 13 dias de execução efetiva, a implementação do robô teria resultado em uma economia de tempo de trabalho estimada em cerca de **10 horas**, não fosse a necessidade de validação manual das execuções nesse primeiro momento, fato esse que não se repetirá, dado que os resultados foram consistentes e pôde-se atestar a confiabilidade da plataforma. Confirmando o impacto positivo da solução na eficiência operacional do escritório, os *feedbacks* foram extremamente positivos por parte dos usuários/colaboradores, que participaram ativamente durante o processo dando sugestões e se dispondo a validar os resultados.

5 Conclusão

Este capítulo apresenta o fechamento do trabalho, consolidando as principais ideias e resultados obtidos. Inicialmente, é realizado um resumo do tema e da problemática que motivou o projeto. Em seguida, discute-se a relevância do estudo para o autor, para a sociedade e para o meio acadêmico. Posteriormente, são apresentadas as conclusões gerais sobre os resultados práticos e o atingimento dos objetivos propostos. Por fim, são elencadas sugestões de melhorias para a plataforma desenvolvida e propostas para trabalhos futuros que possam dar continuidade à pesquisa.

Este trabalho abordou o desenvolvimento de uma plataforma web para o gerenciamento e execução de *scripts* de Automação Robótica de Processos em um escritório contábil de pequeno porte em franca expansão. A proposta central foi não apenas criar robôs para automatizar tarefas repetitivas, mas construir um ecossistema robusto, aplicando boas práticas de engenharia de software para enfrentar desafios comuns em iniciativas de RPA, como a dificuldade de manutenção, a baixa resiliência dos *scripts* e a carência de mecanismos eficazes de monitoramento e governança. O projeto, portanto, situou-se na interseção entre a automação de processos de negócio, a engenharia de software e as arquiteturas de sistemas distribuídos em nuvem.

A relevância deste projeto pode ser observada sob três perspectivas distintas. Para o autor, representou a oportunidade de aplicar os conhecimentos de Engenharia de Computação em um problema real e de alto impacto prático, consolidando conceitos de desenvolvimento web, arquitetura de microsserviços, computação em nuvem e automação. Para a sociedade, em especial para o ecossistema de pequenos negócios contábeis, o trabalho oferece um estudo de caso sobre a viabilidade de uma solução tecnológica acessível, capaz de aumentar a eficiência operacional e promover um ambiente de trabalho mais estratégico, liberando colaboradores de tarefas manuais. No âmbito acadêmico, o presente trabalho contribui com um estudo prático que documenta a concepção, o desenvolvimento e a validação de uma plataforma de RPA utilizando uma arquitetura moderna e serviços em nuvem, servindo como referência para futuros projetos que explorem a aplicação de tecnologia no setor de serviços.

Ao final do período de desenvolvimento e validação, pode-se concluir que os objetivos propostos foram atingidos com êxito. O objetivo geral, de desenvolver uma

plataforma web funcional para um escritório contábil, foi alcançado com a entrega de um MVP que automatizou com sucesso o processo piloto de "Consulta ao DTE". Os objetivos específicos também foram cumpridos: foi implementada uma arquitetura modular e extensível, utilizando tecnologias como *Django*, *FastAPI*, *Docker* e *Google Cloud Platform*; e foram incorporados mecanismos de registro de logs e acompanhamento de status, essenciais para a governança das automações.

A fase de validação prática, detalhada no Capítulo 4, confirmou teorias apresentadas na fundamentação teórica. Conforme apontado por Carvalho (2020), a automação de uma tarefa de alta frequência, como a consulta ao DTE, resultou em uma economia de tempo de trabalho estimada em 10 horas ao longo de apenas 13 dias de execução, o que reforça o potencial de ganho de produtividade. Além disso, o estudo corroborou os achados de Reis e Faria (2024), ao demonstrar que a adoção de RPA em pequenas empresas enfrenta desafios técnicos significativos, como a alocação de recursos computacionais e a configuração de software em ambientes de contêineres, problemas que foram identificados e solucionados de forma iterativa durante os testes. A instabilidade inicial e sua posterior resolução validaram a importância das práticas de engenharia de software e de uma arquitetura resiliente, como o mecanismo de *retry*, para garantir a confiabilidade da solução.

Apesar de o MVP ter atingido seus objetivos, o projeto possui um vasto potencial para evoluções futuras que podem agregar ainda mais valor à plataforma. Entre as implementações futuras, destacam-se:

- **Agendamento de Execuções:** Desenvolver uma funcionalidade que permita agendar a execução automática dos robôs em horários pré-definidos, eliminando a necessidade de disparo manual.
- **Manipulação de Arquivos:** Implementar um sistema de *upload* e *download* de arquivos, permitindo que automações processem planilhas, documentos PDF ou outros artefatos como entrada, e disponibilizem os arquivos gerados (relatórios, guias, etc.) como saída.
- **Alertas e Notificações:** Integrar o sistema a canais de comunicação como e-mail ou mensageria instantânea para enviar notificações sobre a conclusão de tarefas, alertas de erro ou resumos diários das execuções.

- **Estratégias de *Retry* Avançadas:** Aprimorar o mecanismo de *retry*, implementando estratégias como *exponential backoff*, que aumenta o intervalo entre as tentativas, evitando sobrecarregar serviços externos instáveis.

Como sugestão para trabalhos acadêmicos futuros, a pesquisa poderia ser aprofundada nos seguintes tópicos: um estudo comparativo de desempenho e custos entre a solução proposta e as plataformas comerciais de RPA; e a exploração da integração de técnicas de Inteligência Artificial e *Machine Learning* para automatizar processos menos estruturados, que exigem algum nível de interpretação ou tomada de decisão cognitiva.

Referências

- CABRAL, P. H. D. *Horizonte contábil diante da tecnologia Robotic Process Automation*. UFRGS, 2022.

- CARVALHO, M. F. F. *Automatização por Robô de Software para um Sistema Contábil*. UFRN, 2020.

- DELOITTE. *Automation with intelligence. Deloitte Insights*; Disponível em: https://www.deloitte.com/us/en/insights/topics/talent/intelligent-automation-2022-survey-results.html. Acesso em: 03/05/2025.

- REIS, L. J. P. F.; FARIA, A. M. de. *Desenvolvimento do profissional para automação de processos com uso de tecnologia: uma revisão sistemática da literatura sobre RPA em contabilidade*. Estudo & Debate, 2024.

- ITREX GROUP. *Top RPA Challenges and Ways to Overcome Them*. 2022. Disponível em: https://itrexgroup.com/blog/top-rpa-challenges-and-ways-to-overcome-them/. Acesso em: 05/05/2025.

- PLANEKS. *RPA Challenges*, 2023. Disponível em: https://www.PLANEKS.net/rpa-challenges/. Acesso em: 05/05/2025.

- GARTNER. *RPA Perceptions and Adoption*. 2023. Disponível em: https://www.GARTNER.com/peer-community/oneminuteinsights/omi-robotics-process-automation-rpa-perceptions-adoption-fk2. Acesso em: 05/05/2025.

- DJANGO. *Django documentation*. 2025. Disponível em: https://www.djangoproject.com/. Acesso em: 11/05/2025.

- FASTAPI. *FastAPI documentation*. 2025. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 11/05/2025.

- SELENIUM. *The Selenium project and tools*. 2025. Disponível em: https://www.selenium.dev/documentation/en/introduction/the_selenium_project_and_tools/. Acesso em: 11/05/2025.

- FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. 2000. Tese (Doutorado em Ciência da Computação e da Informação) - University of California, Irvine, Irvine, 2000. Disponível em: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm. Acesso em: 11/05/2025.

- DOCKER. *What is a Container*. 2025. Disponível em: https://www.docker.com/resources/what-container/. Acesso em: 12/05/2025.

- GOOGLE CLOUD. *Google Cloud Documentation*. 2025. Disponível em: https://cloud.google.com/docs/. Acesso em: 12/05/2025.

- VALENTE, M. T. *Engenharia de Software Moderna*. 2022. Disponível em: https://engsoftmoderna.info/. Acesso em: 15/05/2025.

- RICHARDSON, *C. Microservice patterns: With examples in java*. Nova Iorque, NY, USA: Manning Publications, 2018.

- RECEITA-RS. O que é o Domicílio Tributário Eletrônico (DTE). Disponível em: https://atendimento.receita.rs.gov.br/o-que-e-o-domicilio-tributario-eletronico-dte. Acesso em: 20/06/2025.